

Prague University of Economics and Business

Faculty of Informatics and Statistics

Department of Information and Knowledge Engineering

Study program: Applied Informatics



**EXPLORATIVE ANALYSIS OF THE EFFECTS OF
HYPERPARAMETERS ON Q-LEARNING IN THE
TEST OF ALGORITHMIC IQ**

BACHELOR THESIS

Author: Matyáš Dlouhý

Supervisor of the bachelor's thesis: Ing. Ondřej Vadinský, Ph.D.

Prague, June 2025

Acknowledgments

I would like to express my sincere gratitude to my thesis supervisor, Ing. Ondřej Vadinský, Ph.D., for their invaluable guidance, support, patience and expertise throughout the process of writing this thesis. Their constructive feedback, direction to valuable sources and encouragement were essential to the completion of this work. I would also like to thank my family for their patience, understanding, and steady support throughout the duration of my studies.

Abstract

This thesis explores how Q-learning hyperparameter configurations affect Q-learning performance in the Algorithmic Intelligence Quotient (AIQ) test. Using a dataset of 945 configurations across 100,000 interactions and 10,000 environments within the test, four analytical approaches are applied: regression models, decision trees, time series analysis and association rule mining. The findings show that learning and exploration rate influence short- and long-term performance, with higher values enabling fast initial improvement but lower values leading to stronger later AIQ scores. Upper extreme values of discount factor and eligibility trace decay rates were found to severely hinder performance, especially when combined with high learning rates. The analyses also reveal compensatory relationships between hyperparameters, namely the learning rates, eligibility trace decay rates and discount rates. Extreme upper values of gamma or lambda are not detrimental to performance on their own, but they do result in poor performance when not compensated by a moderate value of the other parameter. Higher learning rates similarly demand smaller eligibility trace decay rates. Finally, a Python script implementing the full suite of analyses was developed, allowing the same exploratory procedure to be applied to new AIQ test data with other agents, opening an avenue for future refinement and research.

Keywords

AIQ Test, Universal Intelligence, Q-learning, Hyperparameters, Exploratory Data Analysis, Regression, Decision Trees, Time Series, Association Rules, Reinforcement Learning Agents

Abstrakt

Tato práce zkoumá, jak konfigurace hyperparametrů Q-learningu ovlivňují výkon Q-learningu v testu algoritnického inteligenčního kvocientu (AIQ). Na základě datového souboru 945 konfigurací v rámci 100 000 interakcí a 10 000 prostředí v rámci testu jsou použity čtyři analytické přístupy: regresní modely, rozhodovací stromy, analýza časových řad a těžba asociačních pravidel. Výsledky ukazují, že alfa (learning rate) a epsilon (exploration rate) ovlivňují krátkodobý a dlouhodobý výkon, přičemž vyšší hodnoty umožňují rychlé počáteční zlepšení, ale nižší hodnoty vedou k silnějším pozdějším hodnotám AIQ. Bylo zjištěno, že extrémně vysoké hodnoty gamy (discount factor) a lambdy (eligibility trace decay rate) výrazně zhoršují výkon, zejména v kombinaci s vysokou alfou. Analýzy také odhalují kompenzační vztahy mezi hyperparametry, konkrétně alfou, lambdou a gamou. Extrémní horní hodnoty gamy nebo lambdy samy o sobě nejsou škodlivé pro výkon, ale vedou k špatnému výkonu, pokud nejsou kompenzovány umírněnou hodnotou druhého parametru. Vyšší hodnoty alfy podobně vyžadují menší lambdu. Nakonec byl vyvinut skript v jazyce Python, který implementuje celou sadu analýz, což umožňuje použít stejný průzkumný postup na nová testovací data AIQ s jinými agenty, což otevírá cestu pro budoucí zdokonalování a výzkum.

Klíčová slova

Test AIQ, univerzální inteligence, Q-learning, hyperparametry, explorativní datová analýza, regrese, rozhodovací stromy, časové řady, asociační pravidla, agenti zpětnovazebního učení

Content

Introduction.....	7
1 AIQ test and Q-Learning.....	9
1.1 Evaluating Artificial Systems Using the AIQ Test	10
1.2 Reinforcement Learning via Q-Learning	13
1.2.1 Hyperparameters of Q-Learning.....	15
2 Methodology and Analytical Methods.....	17
2.1 Obtaining Knowledge Using the CRISP-DM Methodology	17
2.2 Methods Used in the Exploratory Analysis	19
2.2.1 Regression methods.....	19
2.2.2 Decision Trees	20
2.2.3 Time series analysis.....	21
2.2.4 K-means Clustering.....	21
2.2.5 Association Rule Mining.....	21
2.3 Model Evaluation Metrics and Methods.....	22
2.3.1 Root Mean Squared Error.....	22
2.3.2 Coefficient of determination	23
2.3.3 Silhouette score	23
3 Exploratory Analysis Data	24
3.1 Test Parameter Settings	24
3.2 AIQ Test Data.....	25
3.2.1 AIQ Results at an Episode Length of 100,000 Interactions	25
3.2.2 AIQ Test Results per 1,000 Interactions	26
3.2.3 Data Overview with Summary Statistics	26
4 Q-Learning Performance Analysis Using Regression Models.....	33
4.1 Introduction to the Analyses	33
4.2 Applying and Evaluating Regression Models on AIQ Test Data	34
5 Q-Learning Performance Analysis Using Decision Trees.....	38
5.1 Per-Episode Decision Trees.....	38

5.2 Multi-episode Decision Tree.....	44
6 Q-Learning Performance Analysis Using Time Series	47
6.1 Confidence Interval Based Filtering and AIQ Normalization.....	47
6.2 Top Q-learning Configurations by Frequency	48
6.3 Patterns in Agent Performance.....	50
6.3.1 Highly-performing Configurations at the Start of the AIQ Test.....	50
6.3.2 Highly-performing Configurations at Intermediate Stages of the AIQ Test.....	52
6.3.3 Highly-performing Configurations at the Final Stages of the AIQ Test	53
7 Attempting Clustering Analysis to Categorise Q-Learning Configurations	55
8 Association Rule Mining Using CleverMiner	57
8.1 Mining Association Rules Using the Confidence overlap	57
8.2 Mining Association Rules Using Percentile-Based Categorization	59
9 Discussion	61
9.1 Cross-Method Synthesis.....	61
9.2 Regression analysis	62
9.3 Decision Tree Analysis	62
9.4 Time Series Analysis	63
9.5 Association Rule Mining Analysis.....	63
9.6 Reflections on Methodology	64
9.7 Implications and Further Work.....	65
Conclusions.....	66
List of references.....	67
Appendix.....	I

Introduction

In recent years, the topic of artificial intelligence (AI) has become practically impossible to ignore. With breakthroughs in machine learning, deep learning, and other related disciplines¹ leading to a widening range of applications (Makridakis, 2017, pp. 49-50), the use of AI sharply rising. If the current rate of advancement continues, AI may soon affect various socioeconomic factors of our society (Makridakis, 2017, p. 48). The recent development, along with my growing interest in data science, has inspired the topic of this thesis, titled *Explorative analysis of the effects of hyperparameters on Q-learning in the test of Algorithmic IQ*. By conducting an exploratory data analysis (EDA) of the data gathered from the performance of the *reinforcement learning* algorithm *Q-learning* (Watkins, 1989) in the *Algorithmic IQ* (AIQ) test (Legg & Veness, 2011), I intend to gain a deeper understanding of both the employed analytical tools and Q-learning, presenting the analysis' findings to the reader.

The primary objectives of this thesis are to:

- Conduct an *explorative*² analysis of Q-learning hyperparameter configurations within the Algorithmic IQ (AIQ) test.
- Create a script that can be easily adjusted to run the explorative analysis on AIQ experiments with other agents.

To achieve these goals, I intend to use Python and Jupyter Notebook, utilizing libraries of the Conda package to employ machine learning algorithms such as regression, regression trees, clustering, association rule mining, and traditional statistical methods.

The expected contribution will consist of the analysis results along with a Python script that can be applied to other agent data in the AIQ test to identify desirable configurations.

¹ For a small sample of the recent breakthroughs signifying the mentioned research boom, refer to AlphaZero (Silver et al., 2018), AlphaFold (Jumper et al., 2021), GPT-4 (OpenAI, 2023), or GNoME (Batzner et al., 2023) models.

² Meaning that no prior hypotheses are set before analysing and visualizing the relationships between hyperparameter values of Q-learning and the resulting test score.

While this is the first project of such a scale that I have undertaken, I intend to approach it with due effort and commitment to learning. At the same time, I acknowledge that there may be limitations in the range of models I can effectively implement into the analysis, which could have an influence on the overall quality of the analysis and the resulting script.

[Chapter 1](#) introduces intelligence testing as a theoretically well-founded general methods for artificial intelligence evaluation, presenting the AIQ test as an established tool in this area, and describes the reinforcement learning algorithm examined within the thesis, Q-learning. In [Chapter 2](#) I describe relevant analytical methods applied in the analysis. [Chapter 3](#) presents the examined data and outlines the work resulting its generation. [Chapters 4-8](#) are dedicated to the various analyses performed on the data.

1 AIQ test and Q-Learning

The essence of the human mind has been a commonly discussed philosophical topic since antiquity. However, it was only with the adoption of modern education systems at the turn of the 20th century that sufficient societal demand emerged for the development of tests that measure and compare cognitive performance. The resulting academic effort culminated in the development of *Intelligence Quotient*³ (IQ) tests, which serve as widely recognized measures of human intelligence (Legg & Hutter, 2007a, pp. 5-7).

Hidden under the term IQ test, there are several variations, the most commonly used one being the Raven test (Raven, 2003). IQ tests all tackle the issue of quantifying human cognitive performance by placing a person's result along a Gaussian distribution, with the score of 100 being the average.

With the post-war development in digital computing, and later machine learning, the question of *machine* intelligence arose, with Turing defining the first notable test of artificial systems in the form of the *Turing test* (1950). Turing purposefully stepped away from the question of what intelligence is, and instead focused on its behavioural aspects. The Turing test is impractical for the purposes of performance assessment due to its binary result scale - either the machine succeeds, by convincing the examiner that it is human, or it fails. In the following decades, AI researchers proposed several different approaches to designing tests for systems capable of displaying intelligent behaviour (Legg & Hutter, 2007a, pp. 33-38). With advancements being made in the field of learning algorithms, several non-anthropocentric definitions of intelligence were also introduced (Legg & Hutter, 2007b). Legg and Hutter (2007a) have surveyed the then current state of knowledge with the aim of devising a *universal measure of intelligence* applicable to artificial systems, which is the branch of effort leading to the AIQ test, proposed by Legg and Veness (2011), that will be described in [Section 1.1](#).

The AIQ test is not the only test developed to measure machine intelligence, another notable test for this purpose is the *Comprehension test* (C-test) proposed by Hernández-Orallo

³ The use of the term *quotient* in this context is misleading, stemming from the origin of IQ tests, where the score indeed used to be a quotient. (Legg & Hutter, 2007a, p. 7).

(2000). These tests however differ – among other ways – in that the C-test is deterministic, and doesn’t evaluate the learning process, whereas the AIQ test is stochastic and episodic. Therefore, agents that perform poorly at the start of the AIQ test, but can adapt quickly, are not necessarily disadvantaged, unlike in the C-test. Notably, a comprehensive picture of the broad discussion around machine intelligence evaluation was published by Hernández-Orallo (2017).

The AIQ test serves as a valuable evaluation framework for different models and configurations of reinforcement learning agents. [Section 1.2](#) introduces one such agent: *Q-learning* (Watkins, 1989). For a detailed explanation of Q-learning, along with other reinforcement learning agents, see Sutton and Barto (2017).

1.1 Evaluating Artificial Systems Using the AIQ Test

In this section, I explore the universal intelligence measure (Legg & Hutter, 2007a) and its approximation in the form of the AIQ test (Legg & Veness, 2011). This is a test that assigns agents a numerical score ranging from 0 to 100, thus allowing us to compare different agent types and configurations. The AIQ test moves away from reliance on linguistic and anthropocentric tasks, and instead focuses on intelligence as described by the universal intelligence measure, proposed by Legg and Hutter (2007a).

In their work, Legg and Hutter examined several widely accepted definitions of intelligence along with existing anthropocentric intelligence tests. By aggregating many philosophical and scientific attempts at defining intelligence and examining dozens of the existing formulations and evaluation methods, Legg and Hutter abstracted their definition of universal intelligence as a measure of “an agent’s ability to achieve goals in a wide range of environments” (p. 12). They then used this definition to propose the formal universal intelligence measure (pp. 23, 24).

Legg and Hutter deliberately prioritized mathematical precision and elegance over practical computability (p. 27). Their formal expression assumes an agent is being tested over the infinite space of environments through an endless sequence of interactions, and that the respective environment’s results are weighed based on the environment’s Kolmogorov

complexity⁴, thus applying Occam’s razor⁵. This work was later built upon by Legg and Veness (2011), who proposed a metric named the *Algorithmic Intelligence Quotient*. This computable approximation of the universal intelligence measure is shown by Equation 1:

$$\hat{Y}(\pi) = \frac{1}{N} \sum_{i=1}^N \hat{V}_{p_i}^{\pi}, \quad \text{where } \hat{V}_{p_i}^{\pi} := \frac{1}{k} \sum_{j=1}^k r_j, \quad (1)$$

where the approximated AIQ score $\hat{Y}$ of an agent π is the average of its ability to choose appropriate actions as assessed by the empirical value function⁶ $\hat{V}_{p_i}^{\pi}$, which represents the average accumulated reward over k interactions with an environment program p_i from a finite sample of N environment programs.

The authors use the Solomonoff universal distribution (Solomonoff, 1964) for program sampling from a universal Turing machine to generate a finite set of environment programs used in the test with a preference for less complex programs, as shown by Equation 2.

$$M_{\mathcal{U}}(x) := \sum_{p: \mathcal{U}(p)=x} 2^{-l(p)}, \quad (2)$$

where $\mathcal{U}$ is the chosen reference machine and x is a finite string of bits that is the subject of the probability equation. $\mathcal{U}(p) = x$ means that the universal Turing machine $\mathcal{U}$ computes an output sequence that begins with x when it runs program p , and $l(p)$ is the length of p in bits. As the chance of a program being sampled having an additional bit decreases by $1/2$ with every bit, this distribution favours shorter, less complex environment programs, which follows the principle of Occam’s razor (Legg & Veness, 2011, p. 3-4).

It is important to sample environment programs that discriminate agents⁷, as this property shows that the program is indeed evaluating agent’s performance. A preference for shorter programs serving to approximate Occam’s razor puts additional weight on the choice of the programming language, as using a complex language may result in programs with ‘bizarre’ structures, but simple languages such as Lisp may favour certain agents over

⁴ Which is incomputable.

⁵ Occam’s razor is important to ensure agents aren’t being presented with overly complex programs. This is because, according to (Legg & Hutter, 2007a, pp. 20-22), we want the agents themselves to apply Occam’s razor. By presenting them with a sample of environments that are, on average, less complex, we inherently create a test environment where agents applying this principle are more likely to perform well.

⁶ Reproduced directly, in its explicit form, from (Vadinský, 2018, Equation 2).

⁷ In the sense that there is a considerable variance in the results different agents achieve.

others. Noting that a language allowing programs that mimic the real world would be ideal but hard to develop, the authors opted for a modified version of the minimalist BF language. The BF language only uses 8 symbols and is therefore easy to sample from. In their modified version, a ninth symbol ‘%’ was added, which writes a random symbol to the current work-tape cell, making the resulting programs stochastic⁸. A tenth symbol ‘#’ was also added to mark an end of a program. To prevent the possibility of randomly acting agents achieving a high AIQ, the first, randomly chosen bit of every program denotes whether the rewards achieved will be positive or negative. Therefore, such randomly acting agents would have their score converge to 0 (Legg & Veness, 2011, p. 6).

The basic premise of the proposed test is then to have an agent⁹ exposed to an environment receive an observation and choose an action. After any given action taken, the agent receives a reward and a new observation. The agent then chooses another action, and, assuming its internal logic is devised so, it will try to optimize its strategy, i.e. which actions to take depending on the previous interactions with the environment to maximize the cumulative reward received. This purposefully abstract framework can easily be used to describe practically any situation in real life, where learning plays an important role. For example, a driver on a highway is being presented with an observation (the road) and a set of actions (turning the wheel, changing speed,...). In this case, a reward could be thought of as staying on the road and on the correct lane while maintaining appropriate speed. A good driver maintains the correct speed and position better than a bad one, just as ‘smarter’ agents receive higher rewards than the ones less capable.

As Legg and Veness show in their work, the AIQ score achieved by different agents behaves correctly, awarding agents expected to perform better with higher scores (Legg & Veness, 2011, p. 9-10).

In order to improve the suitability of the AIQ test as a general artificial intelligence evaluation method, further work was undertaken by (Vadinský, 2018), who aimed to reduce the number of programs being sampled that were either non-discriminative or contained unclear, convoluted code. To do this, the author performed a semantic analysis and identified multiple syntactic classes of such programs. Converting these syntactic

⁸ This serves to make the environment programs approximations of Markov Decision Processes, as described by (Bellman, 1957). This stochastic behaviour better mimics the real world, and also increases program variety.

⁹ E.g., a Q-learning agent.

classes into regular expressions allowed Vadinský to extend the AIQ test to iteratively overwrite or discard these programs, resulting in a higher quality test at the cost of a more resource-heavy sampling. This revised sampling promises more representative results, however the author noted that there is inherently room for further work in this area.

1.2 Reinforcement Learning via Q-Learning

The *Q-learning* algorithm is one of the many¹⁰ algorithms leveraging *reinforcement learning* (RL). RL is one of the three main paradigms of machine learning, which is split into three distinct branches, alongside *supervised learning*, which uses labelled data, and *unsupervised learning*, which identifies structure in unlabelled data. Mitchell (1997) offers a comprehensive introduction to this field. To characterise reinforcement learning, I will use an informal definition provided by two of its key pioneers:

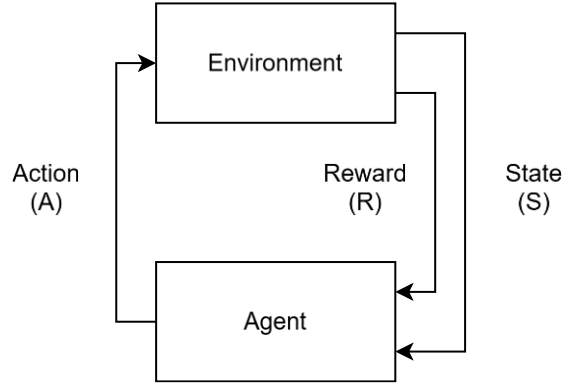
Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them. (Sutton & Barto, 2018)

Legg and Hutter (2007a) used the reinforcement learning framework in the universal intelligence measure. They opted for this framework as it closely represented the action-reward feedback loop used in non-human intelligence tests, within the computational context of artificial intelligence. The work of Legg and Veness (2011) adopted this approach. Therefore, both the AIQ test and RL work with the same notion of machine intelligence and are focused on adaptability of agents in a given environment.

¹⁰ For other traditional RL algorithms, see (Kaelbling, Littman & Moore, 1996).

Figure 1

Diagram showing the reinforcement learning framework



Note. Adapted from (Sutton & Barto, 2018, Figure 6.6).

Figure 1 shows the three main elements of RL, the environment, the agent, and the interactions between these two entities.

The Q-learning algorithm, proposed by Watkins (1989), learns the Q-function, which estimates the optimal action-value function¹¹ (Sutton & Barto, 2018, Section 6.5). This function is represented by the Q-table, where each row represents a state and columns represent actions, each cell containing the Q-value for the specific state-action pair. The Q-value is the expected cumulative reward received by choosing this action and following optimal policy¹² thereafter. This algorithm is defined by,

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)], \quad (3)$$

where $Q(S_t, A_t)$ is the current estimate of the Q-value for state S_t and action A_t , α is the *learning rate*, R_{t+1} is the reward received after taking an action A_t in the state S_t , γ is the discount factor and $\max_a Q(S_{t+1}, a)$ represents the maximum predicted Q-value for the next state S_{t+1} , assuming an *exploitative*¹³ action is taken (Sutton & Barto, 2018, Section 6.5).

¹¹ I.e., which actions lead to highest cumulative reward.

¹² This consists of choosing an action with the maximum Q-value for each successive state.

¹³ As opposed to an explorative action. Exploitative action is the assumed optimal action in a given state, an explorative action is chosen at random.

The algorithm makes use of *Temporal-Difference (TD) learning*¹⁴ (Sutton & Barto, 2018, Chapter 6), a method widely used by RL algorithms, to update its Q-values. TD learning leverages *Temporal Difference Error* (TDE), which is the difference between the observed reward combined with the discounted future value, and the expected Q-value.

Q-learning is an off-policy algorithm. In RL, policies are the mapping of states to actions that guide the agent's decision-making and are reminiscent of stimulus-response rules in the field of psychology. The various existing policy mechanisms range from simple functions and look-up tables (the Q-table in the case of Q-learning), to advanced search algorithms requiring significant computation. They form the core of RL algorithms, as they alone are sufficient for determining behaviour (Sutton & Barto, 2018, p. 6).

What off-policy means, within the context of Q-learning, is that the algorithm does not use its target (i.e. optimal) policy (π) as it interacts with the environment. Instead, it uses the behaviour policy (μ), in Q-learning referred to as ϵ -greedy—a mix of explorative and exploitative actions—to optimise the Q-table and learn the target policy (Sutton & Barto, 2018, p. 103).

As proven by (Watkins & Dayan, 1992, pp. 58-61), with discretely represented state-action values and enough time, Q-learning converges, i.e., its Q-values reach their true optimal values.

1.2.1 Hyperparameters of Q-Learning

Behaviour of the Q-learning algorithm is influenced by the configuration of its hyperparameters, as shown in Equation 3. The listed hyperparameter values will be subject to my analysis:

- *Learning rate* α ¹⁵ represents the weight by which a current state-action value pair in the Q-table is modified. At low values, the algorithm updates the values slowly and at higher values each observation causes a large change to the Q-value. I suspect that a high learning rate may lead to instability in agent's performance, especially in

¹⁴ TD learning is a method shared by all reinforcement learning algorithms.

¹⁵ Referred to as the step-size parameter by Sutton and Barto (2018, Section 1.5).

stochastic environments, as the agent may end up chasing after a reward that appeared at a low probability, leading to worsening overall performance.

- *Discount factor* γ influences how long-term or short-term reward-seeking the policy of the agent will be. Configurations with a high discount factor seek long-term rewards despite lower short-term gains¹⁶ and vice versa (Sutton & Barto, 2018, Section 3.3).
- *Exploration rate* ϵ represents how likely an agent is to choose an exploitative or an explorative action. An exploitative action is one that follows the agent's policy, whereas an explorative action is an off-policy action that allows agent to potentially discover state-action pairs that yield higher rewards. The rate is represented by a decimal value between 0 and 1 corresponding to the percentage chance of an action being explorative. This strategy, called epsilon-greedy, consists of choosing a random action when exploring. Other strategies are used in reinforcement learning, such as upper confidence bound (UCB) action selection, where instead of a random exploratory action, actions with high uncertainty are likely to be chosen (Sutton & Barto, 2018, Chapter 2).
- *Eligibility trace decay rate*¹⁷ λ influences how strongly past state-action pairs are updated (i.e. their Q-value). Eligibility traces track past action-states and update their values, with the weight of these changes decaying with time as further actions are taken by the agent. Q-learning uses Watkins' $Q(\lambda)$, where traces are reset when a non-greedy action is selected (Sutton & Barto, 2018, Section 7.6).

¹⁶ An intuitive real-life example of this would be investing money to grow its value instead of spending it on, for example, luxury goods of diminishing value.

¹⁷ This hyperparameter is the only one listed that is not ubiquitous to all Q-learning algorithms.

2 Methodology and Analytical Methods

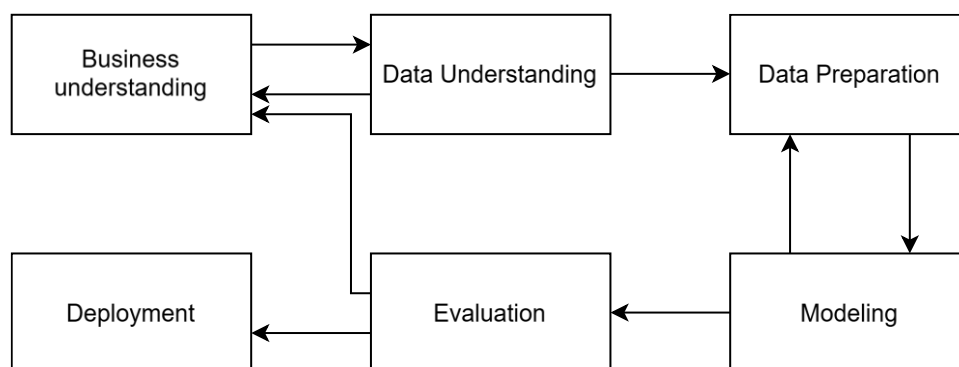
In the previous chapter, I have described subjects forming the data of my analysis, i.e. the AIQ test and the algorithm that is deployed within the test's framework. This chapter introduces analytical tools that I intend to use to obtain knowledge regarding this data, starting with the CRISP-DM methodology (Chapman et al., 2000), an important guideline for forming the workflow of the analysis, followed by the analytical methods used in the analysis, as well as look at the relevant methods and metrics used later in the analysis. These methods and metrics are implemented in Python using appropriate libraries and packages in the explorative data analysis performed in [Chapters 4-8](#).

2.1 Obtaining Knowledge Using the CRISP-DM Methodology

For the purposes of guiding the workflow of the analysis, I have opted to adapt the Cross-industry Standard Process for Data Mining (CRISP-DM) methodology (Chapman et al., 2000), an industrial standard for the process of data mining developed in Europe in the late 90s by a consortium of companies (Mařík, Štěpánková & Lažanský, 2003, pp. 357-358). CRISP-DM describes data mining as an iterative process, composed of six phases. Despite its origin in a business context, its structured approach is easily adaptable for the purposes of a scientific analysis.

Figure 2

Diagram showing the CRISP-DM phases with arrows indicating the workflow.



Note. Adapted and translated from Mařík, Štěpánková & Lažanský (2003, Figure 11.2).

I will now briefly go over each of the six phases highlighted in Figure 2:

- Business Understanding consists of defining the objectives and a project plan of the analysis from a business perspective, as, in its original form, the CRISP-DM methodology focuses on commercial data mining. In general, its goal is to identify what objectives the customer wants to accomplish (Chapman et al. 2000, p. 14). For my purposes, business understanding translates to defining research objectives for the goals of the exploratory analysis of Q-learning performance, for example, discovering which configurations show promising results at different stages of the AIQ test.
- Data Understanding is concerned with the collection, description and verification of project data. Data Understanding can lead the project back to Business Understanding: for example, in the case of uncovering many missing values in the available data, the original goals may need to be adjusted so that they remain achievable. This step of CRISP-DM is covered by [Chapter 2](#).
- Data Preparation phase consists of preparing the raw data, resulting in clean datasets, ready to be used for the purposes of the subsequent data mining. It deals with the questions of which data to use, how to handle missing or corrupted values, whether to merge data, and which derived attributes should be added. (Chapman et al. 2000, pp. 21-22).
- Modelling consists of model selection, employment of quality verification techniques, model building involving parameter tuning, and model assessment (Chapman et al. 2000, pp. 24-25). This crucial step is captured in [Chapter 4](#).
- Evaluation assesses the degree to which the models meet the initial objectives, determining whether there is some reason why a model is deficient in regard to the goals set by the Business Understanding phase. The evaluation further concerns the model results themselves, potentially providing useful insights for future directions or the refinement of the current project (Chapman et al. 2000, p. 26).
- Deployment is the implementation of successfully evaluated models in accordance with the original goals set by the Business Understanding phase (Chapman et al. 2000, pp. 21-22). This can range from a model being deployed for a daily use within a business process, or in the case of academic work, in exporting such model for further research or use.

As stated and highlighted previously, the iterative character of CRISP-DM allows for the backtracking between project phases, which appears to offer better results than traditional waterfall models.

During the process of the analysis, the CRISP-DM methodology was used as a non-binding methodology for the workflow, resulting in a number of iterations of various models, which were built upon and improved after self-evaluating my work and collecting valuable feedback from my supervisor.

2.2 Methods Used in the Exploratory Analysis

In this section I will go over all analytical methods used in the EDA, performed in [Chapters 4-8](#). Each method is first introduced on a general level, followed by a mention of the relevant Python implementation. Several of the described methods employ supervised and unsupervised machine learning via algorithms that find patterns from data to improve their performance. In supervised learning, models learn from labelled examples to predict the target variable based on independent variables. Tasks in this category can be split between regression, where the target variable is continuous, and classification, where the target variable is categorical. In contrast, unsupervised learning, used in clustering, aims to find patterns in unlabelled data (Géron, 2022, Chapter 1).

2.2.1 Regression methods

As regression models are one of the most common analytical tools to find trends in data analyses, I intend to employ a range of regression models in [Chapter 4](#). The goal of regression methods is to model the relationship between a numerical target variable and one or more independent numerical variables using a given function. These methods estimate the observed data using parameters that result in the approximate optimal fit for

the data. In its simplest form, linear regression with one independent variable¹⁸, the aim is to find the values of parameters q_1 and q_2 for:

$$y = q_1 x + q_0 + \varepsilon, \quad (4)$$

Where y is the target variable and x is the independent variable. A common way to find optimal parameter values is to use the least squares method (Berka, 2003, pp. 49-51).

Linear regression only sufficiently approximates linear relationships and has a high degree of bias when applied on data with higher complexity. For more complex relationships between the target and independent variables, more complex regression models with exponential terms, such as quadratic, cubic, etc., may better fit the observed data.

While regression analysis is usually conducted for predictive purposes, in the context of this thesis, it serves as a tool for visualising potential relationships between hyperparameter values and AIQ score. In [Chapter 4](#) of the thesis, I use it to interpolate between the limited number of discrete hyperparameter values. Within the EDA, it is implemented by the Scikit-learn LinearRegression object.

2.2.2 Decision Trees

A decision tree is an easily interpretable model used for classification and regression tasks. It employs one of several algorithms using the divide and conquer method, meaning that data is being divided into smaller and smaller subsets based on splits chosen for the independent variable values. This top-down approach produces branching paths representing a tree model. For classification trees, Gini index or entropy are commonly used to decide which independent variable value will be used to split the tree (Berka, 2003, pp. 86-87).

A regression tree is a type of decision tree used for predictive modelling with a continuous target variable, making it ideal for the exploratory analysis. It works by recursively sub-setting the data, with the goal of minimizing the variance within each subset.

¹⁸ I.e. univariate linear regression. Regression models with more than one independent variable are referred to as multiple.

In [Chapter 5](#), a Decision Tree Regressor from the sci-kit library is used in order to model the relationship between hyperparameter values and the AIQ score. This implementation uses the MSE as a criterion to determine optimal split points in the data.

2.2.3 Time series analysis

Time series analysis is a powerful visualisation tool for examining temporal development of a given variable, consisting of time series plots that allow analysts to observe patterns in data over time. This sets it apart from the previously mentioned methods, which are focused on capturing relationships at a single point in time. A time series thus enables the study of trends and stability over the course of the AIQ test.

I employ the time series analysis in [Chapter 6](#) as a way to study the AIQ convergence of Q-learning configurations throughout the test, focusing in particular on high-performing configurations at various stages of the test.

2.2.4 K-means Clustering

K-means clustering is an unsupervised machine learning algorithm used to partition data into k distinct clusters, each with its own *centroid*, a point representing the mean variable values of all members of the cluster. It assigns each point of data to the nearest centroid, iterating the centroids to minimise intra-cluster variance.

Within the thesis, in [Chapter 7](#), K-means clustering is used to segment hyperparameter configurations based on their performance and to identify common properties among them. To identify the optimal number of clusters, the *silhouette score* is used, as the value k needs to be specified manually. The silhouette score is described in [Section 2.3.3](#).

2.2.5 Association Rule Mining

Association rule mining is a data analysis technique aimed at uncovering interpretable relationships (rules) expressed in an “if-then” format. These rules are evaluated using a 4-fold contingency table, which expresses the occurrence of the antecedent (condition) and succedent (result) in a 2x2 matrix. This structure allows for additional metrics, namely support and confidence, to be calculated in order to express the strength of discovered rules. Support is the number of times in which both the antecedent and succedent occur, also referred to as coverage. Confidence, or accuracy, is the proportion of instances with the antecedent where the succedent is also observed (Witten & Frank, 2005, Section 3.4).

In [Chapter 8](#), I employ the 4ft-miner, a GUHA procedure (Berka, 2003, p. 281). 4ft-miner is one of the data-mining procedures of the CleverMiner Python package (Masa & Little Big Company, 2025).

2.3 Model Evaluation Metrics and Methods

When creating models using the methods described in further sections¹⁹, my goal is to strike a balance between bias and variance, or put in a slightly different way, the balance between underfitting and overfitting. Models with high bias make simplistic assumptions about the actual trends in the data, for example, assuming that the observed data is linear, when it more closely resembles a quadratic relationship. This would be referred to as underfitting. On the other hand, a model such as a regression tree with a large depth, or a high-degree polynomial regression curve, tend to display high variance, overfitting the provided data and thus also failing to accurately model the actual relationships. The last part of the variance-bias issue, or the generalization error, is the irreducible error, caused by outliers in the data, which can only be reduced by ensuring that the original data has been measured correctly (Géron, 2022, Chapter 4). The following metrics help in identifying the point at which the balance between bias and variance has been achieved or when the model suffers from either of the two influenceable components of the generalization error.

2.3.1 Root Mean Squared Error

The root mean squared error represents one of the most common evaluation metrics in regression tasks. It is defined as the square root of the mean of squared distances between the predicted and actual values of the model, placing a bigger weight on large errors. This makes RMSE less ideal for data with significant outliers, compared to metrics such as the mean absolute error (MAE) (Géron, 2022, Chapter 4).

¹⁹ Outside of modelling regression curves, which I present simply as descriptive visualisation tools for the observed data, without the goal of using them, or enabling a potential user to use them, for any predictive analytics.

2.3.2 Coefficient of determination

The coefficient of determination, in data analytics often referred to as the R^2 score, is a valuable evaluation metric used commonly to evaluate regression models. It can be thought of as the proportion of data variance in the target variable that is predictable based on the independent variables, it compares how much better the model describes the data relative to simply predicting the mean of the target variable. R^2 is particularly simple to interpret, as value of 1 indicates a perfect fit, whereas values below 0 indicate that the model performs worse than using the target variable's average (Chicco, Warrens & Jurman, 2021).

2.3.3 Silhouette score

The silhouette score is a metric used to identify optimal number of clusters in K-means clustering. While more computationally expensive than the heuristic elbow method²⁰, it offers a more mathematically grounded, robust approach for determining K, i.e., the number of clusters. It is the average silhouette coefficient²¹ across all instances of data.

Values of the silhouette score range from -1 to +1. Coefficient values near +1 indicate that an instance is close to its cluster centroid, while values near 0 indicate an instance is close to the cluster boundary. Finally, sub-zero values indicate an instance may have been assigned to the incorrect cluster (Géron, 2022, Chapter 9). The mean of the coefficient sum indicates the overall quality of all clusters for a given K, allowing for quick identification of optimal K using a simple for-loop logic, as the silhouette score values typically gradually increase before reaching a peak signifying the value has been found.

²⁰ A heuristic method based on plotting cluster count k on inertia - within-cluster sum of squares (WCSS). A decision for optimal k is based on empirical observation of this graph, specifically on identifying the “elbow” in the plot line (Géron, 2022, Chapter 9).

²¹ Which is the difference between the average distance to inter-cluster points and the average distance to the nearest neighboring cluster, divided by the greater of the two values, i.e. $(b - a) / \max(a, b)$ (Géron, 2022, Chapter 9).

3 Exploratory Analysis Data

This chapter describes the data being analysed in the [exploratory analysis](#). All data comes from AIQ test results of Q-learning agent configurations conducted by Vadinský, using test execution parameters and program sample generation parameters from (Vadinský & Zeman, 2023). The analysis focuses on a set of 945 Q-learning agent configurations in an AIQ test at an episode length of 100,000 interactions, with intermediate results recorded at every 1,000 interactions.

3.1 Test Parameter Settings

For the environment program sampling of the AIQ test, the SEP-ext and SDP program optimization methods using semantic analysis, as described by Vadinský (2018), were used to iteratively filter out and replace pointless program code, improving the test's performance. The following parameters were used to generate a sample of programs used:

```
-s 200000 --improved_optimization --improved_discriminativeness
```

Where `-s 200000` specifies the number of programs to be generated, and `--improved_optimization`, `--improved_discriminativeness` denote the use of optimization methods in the process of sampling, with improved optimization aimed at removing redundant program codes and, more notably, improved discriminativeness increasing the number of discriminative programs, improving the resulting test's quality.

As for the AIQ test itself, the following test execution parameters were run:

```
--log --verbose_log_el --log_agent_failures -r BF,5,1 -l 100000 -s 10000
```

where `--log` enables basic logging of the test process, and `--verbose_log_el` enables logging for agent performance throughout the test at regular intervals. `--log_agent_failures` records failures encountered by the agents during the test, though this setting is not active in approximately 19% of all configurations. `-r BF,5,1` specifies the reference machine used, in this case modified BF, as mentioned in [Section 1.1](#), using a 5-symbol tape and 1 observation cell. `-l 100000` denotes the total number of interactions the agents undergo on the sample size of the 10000 environment programs, denoted by `-s 10000`.

3.2 AIQ Test Data

I have been provided with 2 datasets concerning the agent hyperparameter configurations and the AIQ scores obtained by these configurations. I will refer to the smaller set containing the results of the final episode of the test as AIQ results, at an episode length of 100,000 interactions. The larger set, containing the same configurations and results for each 1000 interactions, will be referred to as AIQ Test results per 1,000 interactions. These datasets contain all hyperparameter data used for the analysis in Chapter 4.

3.2.1 AIQ Results at an Episode Length of 100,000 Interactions

This dataset contains the agent information for the final 100,000th interaction test results²², it therefore cannot be used to show the learning capability of Q-learning over time, only their ultimate scores after the end of the test. However, due to its fractional size, this dataset allows for a more computationally efficient analysis in comparison with the full dataset. The dataset represents a matrix where each row represents individual Q-learning configurations and AIQ test results. Table 1 shows the first three rows of this dataset, each representing one Q-learning agent configuration, and all columns relevant to the analysis²³, recording hyperparameter values and AIQ test results. The full dataset contains 945 rows and 13 columns in total.

Table 1

Sample of the AIQ test result dataset at the episode length of 100,000.

AIQ	HCI ²⁴	SD	λ	α	ϵ	γ
61.8	0.4	22.4	0.0	0.3	0.005	0.6
62.1	0.4	22.7	0.0	0.3	0.005	0.7
62.1	0.5	23.2	0.0	0.3	0.005	0.8

²² This is an arbitrarily set limit for the scope of the test, the word final denotes that the test has run to its predefined conclusion.

²³ Columns with constant or irrelevant values were omitted, such as the ones denoting the reference machine used and the RL model being tested.

²⁴ Half of the confidence interval (CI).

3.2.2 AIQ Test Results per 1,000 Interactions

This dataset contains agent information at each 1,000-episode increment of the test, offering a look at agent performance through time. This dataset offers an insight on the agent performance using time series analysis, as well as providing snapshots at different stages of the test, useful for identifying promising, fast-learning hyperparameter configurations for cases where there may be less time to train Q-learning models. Table 2 provides a sample of this dataset, excluding the same, analysis-wise irrelevant attributes as the dataset containing the AIQ results at the episode length of 100,000.

Table 2

Sample of the AIQ Test Results per 1,000 Episodes dataset.

AIQ	HCI	SD ²⁵	λ	α	ϵ	γ	Episodes
43.0	0.5	25.6	0.0	0.3	0.005	0.6	1000
43.9	0.5	25.1	0.0	0.3	0.005	0.7	1000
43.4	0.5	25.1	0.0	0.3	0.005	0.8	1000

As can be seen when comparing Table 1 and Table 2, the AIQ scores achieved by configurations at the first 1000 interaction episodes are noticeably lower than those achieved after the final episode, confirming the reinforcement learning algorithm allowed the configurations to improve their behaviour throughout the test run.

3.2.3 Data Overview with Summary Statistics

This section provides summary statistics followed by basic charts such as histograms and scatter plots focused on the Q-learning AIQ scores and hyperparameters, respectively. The AIQ test dataset at episode length of 100,000 interactions consists of 945 Q-agent configuration test data, consisting of their respective AIQ scores, half confidence intervals and standard deviation values, reference machine data, number of actions available to the agent, test discount factor, episode length, agent architecture and the initial Q-value. In the code, this dataset is imported as a Pandas DataFrame. **Table 3** shows a sample of the data, excluding columns with values irrelevant to the scope of my analysis, such as the reference

²⁵ Standard deviation.

machine used or the number of actions the agent was able to take. The full dataset contains 945 rows and 13 columns, all of which were kept for easier output generation.

Table 3

Head (first 5 rows) of `df_results_summary` DataFrame, a subset of `df_results`. 5 sample rows of data used in the EDA, with each row representing one Q-learning agent configuration

AIQ	HCI	SD	Lambda	Alpha	Epsilon	Gamma
61.8	0.4	22.4	0.0	0.3	0.005	0.6
62.1	0.4	22.7	0.0	0.3	0.005	0.7
62.1	0.5	23.2	0.0	0.3	0.005	0.8
62.3	0.5	23.7	0.0	0.3	0.005	0.9
61.7	0.5	23.9	0.0	0.3	0.005	0.95

I will start by looking at the summary statistics of this dataset. In Python, this is best done using Pandas `.describe()` method, the output of which is shown in Table 4.

Table 4

The `.describe()` method output on `df_results_summary` subset, with the 'Count' row excluded

	AIQ	HCI	SD	Lambda	Alpha	Epsilon	Gamma
Mean	57.060	0.442	22.981	0.604	0.500	0.021	0.848
Std²⁶	8.636	0.057	2.218	0.371	0.163	0.013	0.142
Min	22.600	0.400	20.000	0.000	0.300	0.005	0.600
25%	57.100	0.400	21.400	0.300	0.300	0.010	0.700
50%	59.900	0.400	22.300	0.700	0.500	0.020	0.900
75%	61.700	0.500	23.800	0.950	0.700	0.030	0.990
Max	63.900	0.600	31.000	0.995	0.700	0.040	0.995

At least 75% of configurations fall into an AIQ interval of [57.1, 63.9], meaning that the majority of configurations performed at a similar level²⁷ near the top boundary of the overall AIQ score. This is likely a consequence of the length of the test, as agent performance by the 100,000th interaction appears to mostly plateau. Indeed, as can be seen in the work of Vadinský and Zeman (2024, Figure 1), AIQ performance of most configurations begins

²⁷ Comparatively to all configurations being evaluated by the AIQ test.

to stabilize, achieving only marginal improvements, long before this stage. The rather narrow AIQ interval highlights upcoming challenges in differentiating best performing configurations.

The summary also offers a rough insight into the distribution of hyperparameter values, which I will further explore in Table 5, using the Python script to obtain each discrete hyperparameter value, later using scatter plots showing hyperparameter value distribution on AIQ score.

Table 5

Unique hyperparameter values across the dataset.

Hyperparameter	Unique value count	Values
Lambda	9	{0.0, 0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99, 0.995}
Alpha	3	{0.3, 0.5, 0.7}
Epsilon	5	{0.005, 0.01, 0.02, 0.03, 0.04}
Gamma	7	{0.6, 0.7, 0.8, 0.9, 0.95, 0.99, 0.995}

I originally worked with a smaller dataset, containing 620 agent configurations. After identifying the discrete values of each hyperparameter, I found that 325 additional configurations, which would complete the space of all possible combinations, were missing. This led my supervisor to run the AIQ test again on the additional configurations, ensuring all combinations of the hyperparameter values are included within the analysis.

To gain an insight into how the whole set of configurations learns over time, I have created AIQ histograms for every 1,000-interaction step. Figure 3 contains 4 histograms from different stages of the test. I have chosen episode lengths that show 4 notable stages of the AIQ test, i.e. the very beginning, the point at which the main trends begin to establish itself, the halfway point of the test and the final recorded stage, respectively. Below is an overview of my insights for each histogram chosen and visualised:

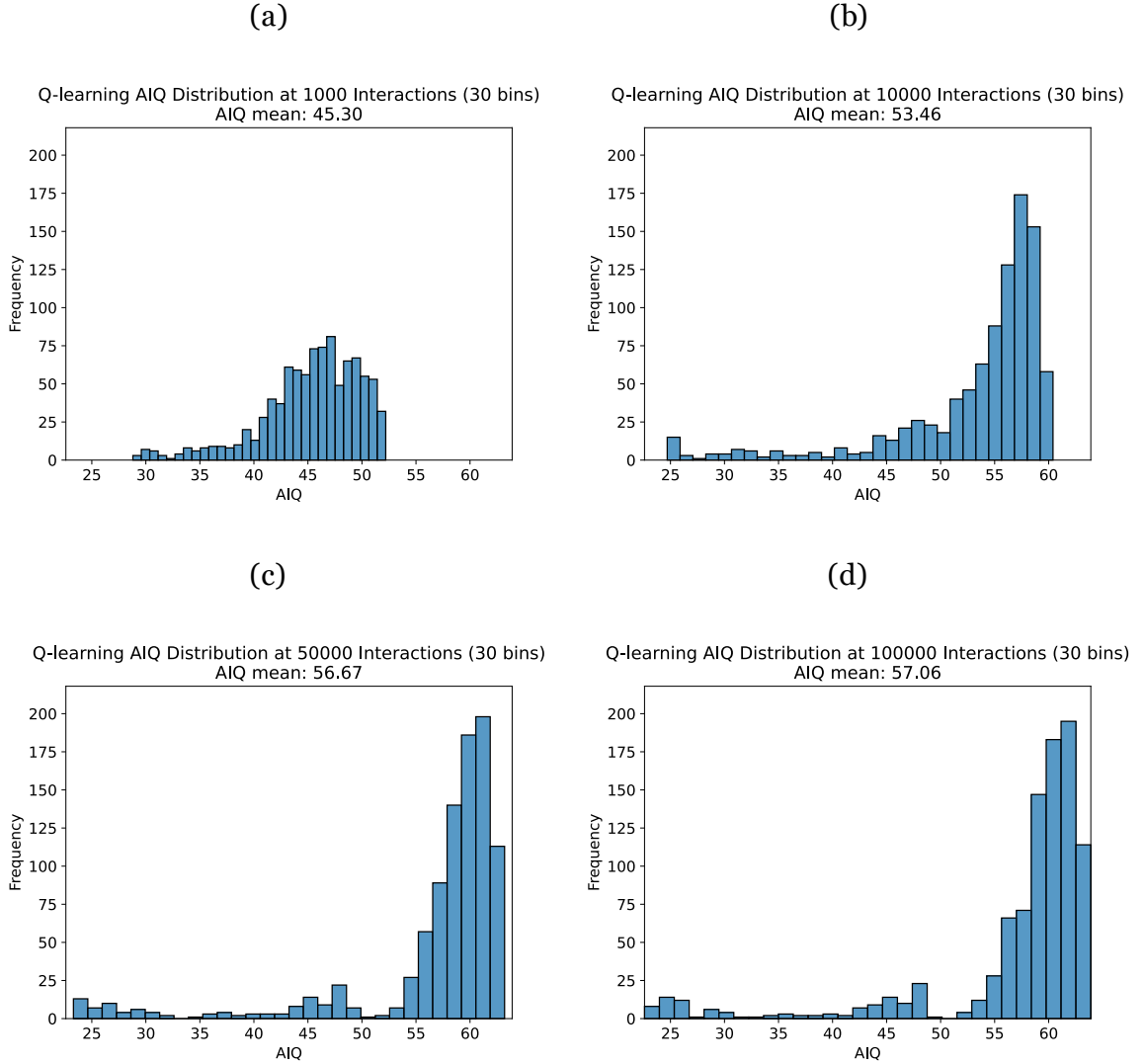
- Figure 3(a) shows AIQ distribution at episode length of 1,000 interactions. The distribution shows a slight right skew, but remains approximately symmetrical. This indicates some configurations have started to outperform others. There is also a small group of configurations forming a left tail.
- Figure 3(b) shows AIQ distribution at episode length of 10,000 interactions. By this point of the test, a major trend in agent performance that carries itself to the end is already apparent – a pronounced right tail consisting of roughly half the

configurations, opposed by a significantly smaller left tail, suggesting majority of configurations are comparatively similar and well-performing, while a small subset is gradually achieving lower AIQ scores than at the start of the test.

- Figure 3(c) shows AIQ distribution at episode length of 50,000 interactions. The shape of the distribution is even more right-heavy than at the previous stage. The mean AIQ value across all configurations is steadily growing, by over 3 points during the 40,000 interactions but it is clear that the configurations are beginning to converge.
- Figure 3(d) shows AIQ distribution at episode length of 100,000 interactions. The distribution remains largely identical, with only marginal improvement in the mean AIQ score. This indicates convergence across most configurations.

Figure 3

Distributions of AIQ scores across episode counts for Q-learning configurations



The left tail observed within the histograms represents worsening configurations. Upon inspecting the 37 configurations that scored lower AIQ values than the minimum AIQ recorded at the start of the test (28.8), and 20 others that performed worse at the end of the test than at the start of it, I found common threads within their hyperparameter configurations. Notably, these configurations all display a combination of high eligibility trace decay rates ($\lambda \geq 0.95$), high learning rates ($\alpha \geq 0.5$, with a mean of 0.63), and high discount factors ($\gamma \geq 0.95$). These configurations update Q-values at high rates, while seeking long term rewards and awarding a high degree of past actions to obtained rewards. The exploration rates of these configurations cover the whole range of discrete values, suggesting no influence of various exploration rates on the low AIQ results.

For a better understanding of the value distributions of hyperparameters, I created scatter plots of each hyperparameter in relation to the AIQ score at 100,000 interactions (Figure 4). Figure 5 shows line charts for each scatter plot, showing the mean AIQ score for each unique hyperparameter value observed and an error bar for the 95% confidence interval. Empirical observation of the scatter plots suggests that lambda and epsilon exhibit value ranges with stable, relatively strong AIQ results: approximately [0.3, 0.9] for lambda and [0.6, 0.9] for gamma. For alpha, value of 0.3 displays the narrowest confidence interval as well as the highest AIQ mean. The wide confidence interval across all epsilon values may indicate that it has a much smaller impact on the overall agent performance. However, it is too early to draw any conclusions as different epsilon values may very well play an important role for various undiscovered hyperparameter combinations.

Figure 4
Scatter plots of each hyperparameter on AIQ

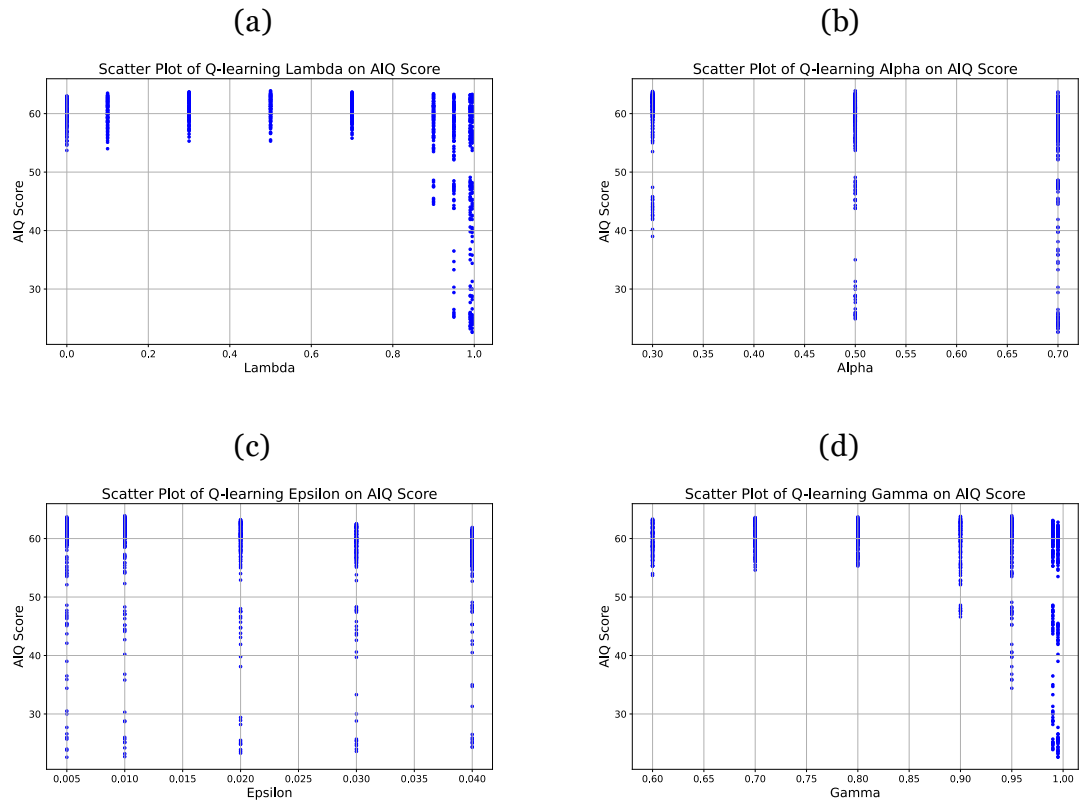
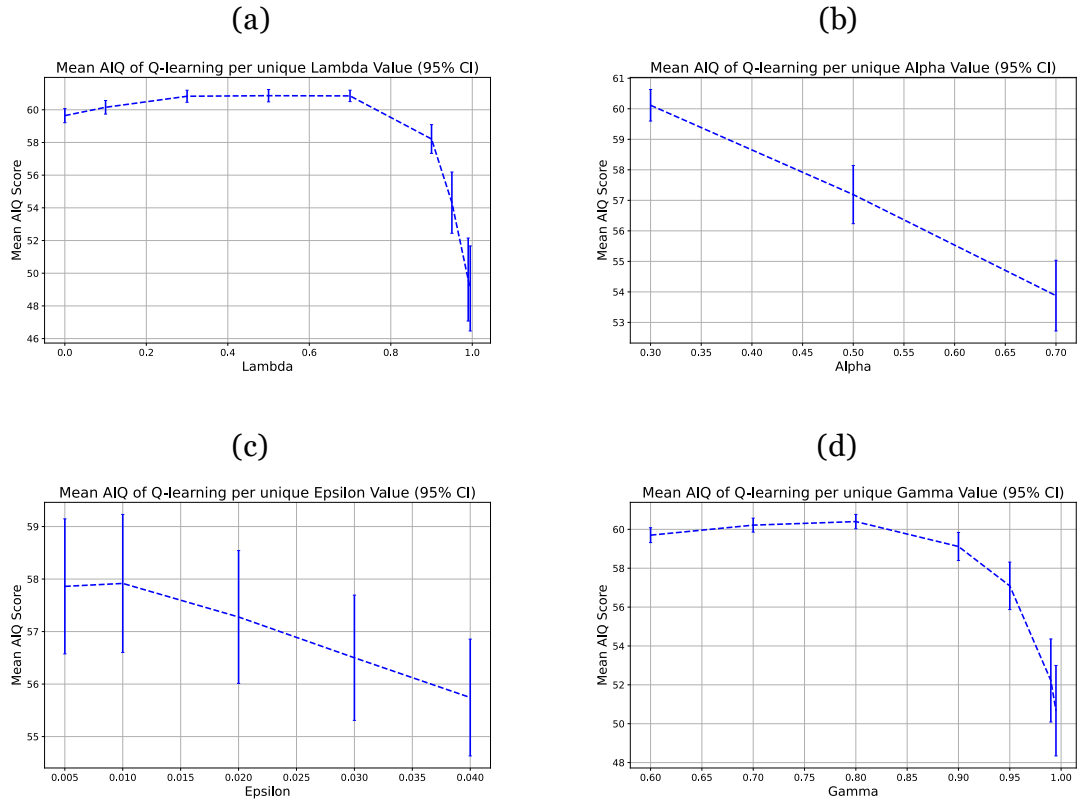


Figure 5

Line charts of each hyperparameter with mean AIQ values and 95% confidence intervals



Having completed this part of the analysis, I gained valuable preliminary insights into the dataset, notably about the hyperparameter and value distribution and agent performance at the end the AIQ test. Analyses conducted in later chapters employ methods that aim to map the relationships between hyperparameter configurations and the AIQ score.

4 Q-Learning Performance Analysis Using Regression Models

This first purely analytical chapter of the thesis establishes the outline of the EDA within [Section 4.1](#). [Section 4.2](#) is devoted to the first employed analysis, using regression models to explore relationships between hyperparameter values and the AIQ score.

4.1 Introduction to the Analyses

The exploratory analysis described in the upcoming chapters is conducted using Python (Python Software Foundation, 2024), Jupyter Notebook (Kluyver et al., 2016) and various libraries of the Conda package for data analysis (Anaconda Software Distribution, 2020), most notably Sci-Kit (Pedregosa et al., 2011), Matplotlib (Hunter, 2007), Pandas (The pandas development team, 2020) and NumPy (Harris et al., 2020).

The input data is composed of two datasets in `.csv` format. The first one, imported and transformed into the 'df_results' DataFrame, only contains the AIQ test results at an episode length of 100,000 interactions. The second dataset, represented by the 'df_results_full' DataFrame, contains these results for each 1000 interaction step, thus being 100 times larger than the former and more computationally expensive to work with. An argument could be made for only importing the larger of the two and subsetting it within the code. However, as I have been provided with both datasets separately, I opted to work with each of the two.

With the goal of identifying optimal hyperparameter values at various distinct stages of the AIQ test, the analysis first focuses on summary statistics and basic charts, followed by the implementation of a scope of methods to create models aiming to describe interesting relationships and trends. Finally, I aim to draw conclusions from empirical observations and model evaluation metrics in conjunction with theoretical knowledge about individual hyperparameters, established in [Section 1.2.1](#).

Due to the thesis' scale constraints, only a small subset of the recorded results is displayed within the work itself. For the full results, see [Appendix](#).

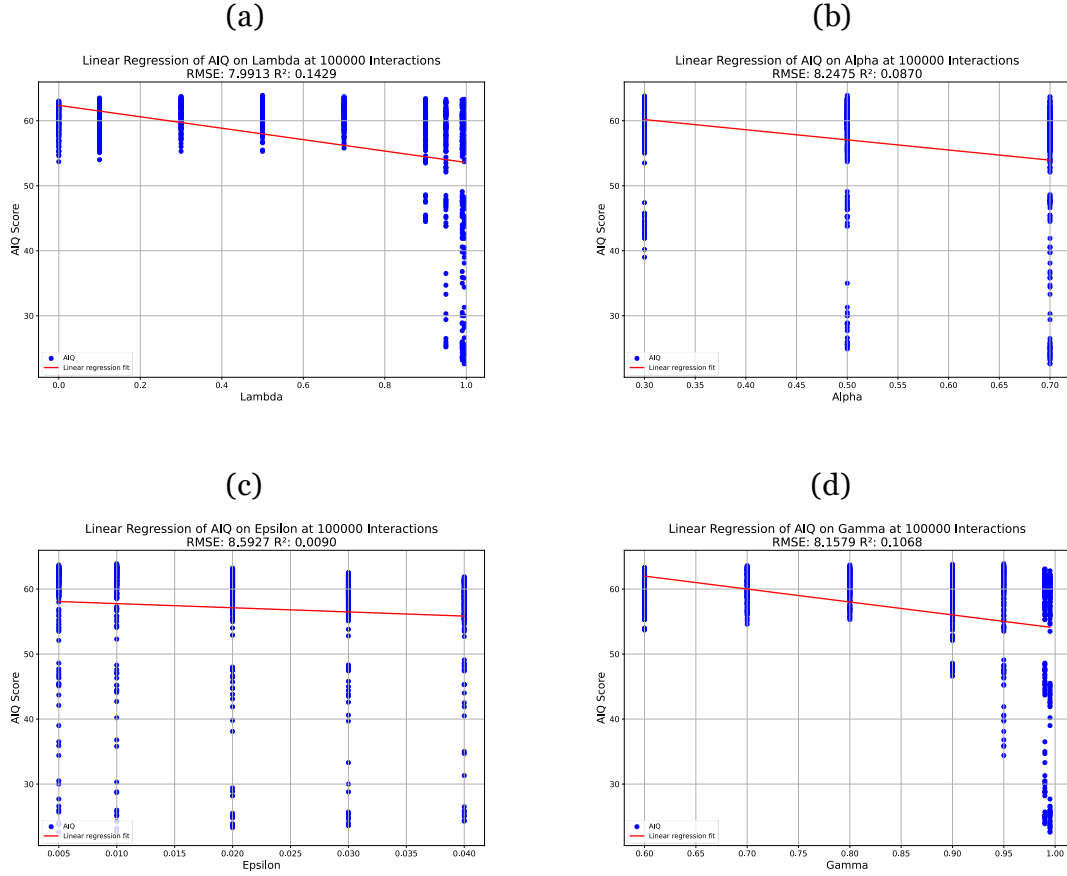
4.2 Applying and Evaluating Regression Models on AIQ Test

Data

This section examines regression models of the relationships between hyperparameter values and AIQ scores. Figures 3 and 4 presented in Section [3.2.3](#) helped highlight the discrete character of the sampled hyperparameters, this property of the available dataset inherently limits the knowledge that can be gathered about the Q-learning configuration search space, as there is no testing data for missing value ranges and their combinations. However, using regression, we can interpolate for these missing values and propose optimal hyperparameter values by finding the maximum of the regression function, representing maximum AIQ score and the corresponding parameter value, albeit avoiding bias in simple regression models proves difficult. I performed two regression analyses, first a linear regression, using which I was able to propose a potential high-performing configuration with $\lambda = 0$, $\alpha = 0.3$, $\epsilon = 0.005$, and $\gamma = 0.6$. (Table 6), and later a quadratic regression configuration, with $\lambda = 0.3729$, $\alpha = 0.3$, $\epsilon = 0.005$, and $\gamma = 0.7378$ (Table 7). Both of these predicted high-performing configurations suggest that for configurations which are allowed to learn over a long term, in the case of the performed test for 100.000 interactions, values at the lower end across all hyperparameters are preferred. Notably, in the case of the exploration rate, the lowest discrete value contained in the dataset seems the most promising in training the best-performing configurations, according to both types of regression models. Figures 6 and 7 show the corresponding regression models.

Figure 6

Linear regression models (univariate), with root mean squared error and coefficient of determination metrics

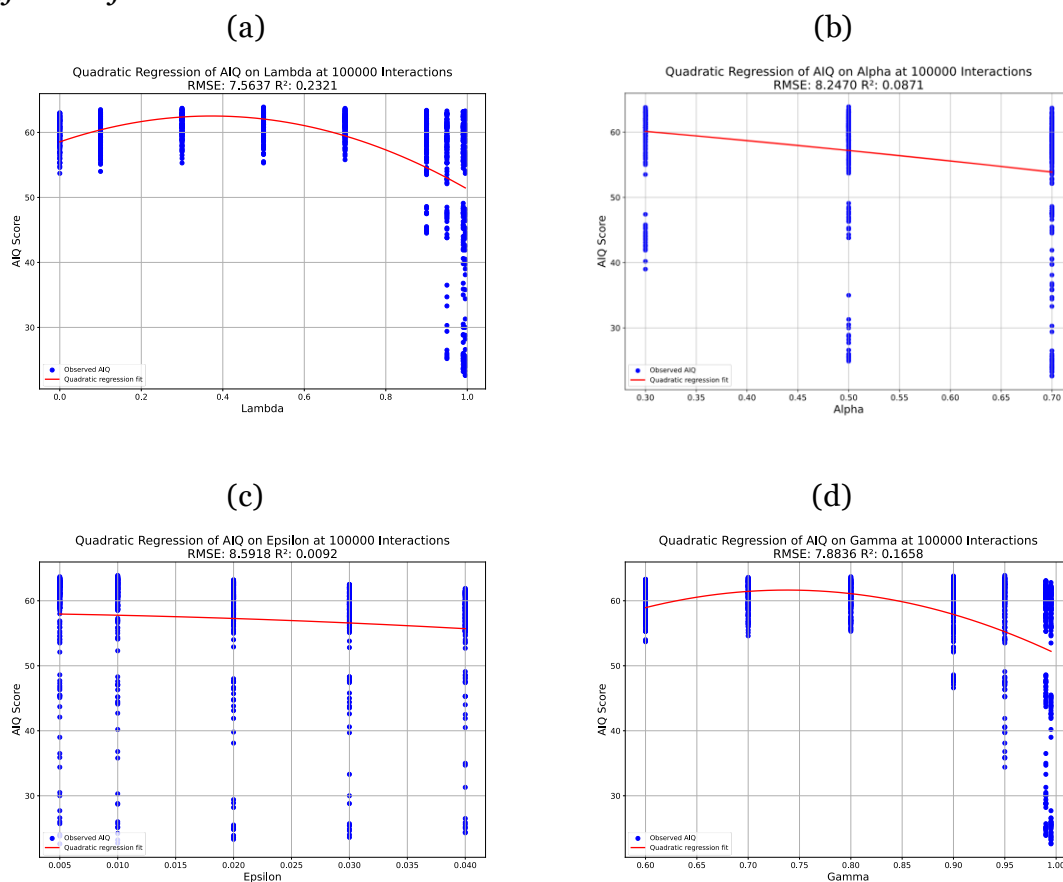
**Table 6**

Linear regression line maxima within the observed hyperparameter range at 100,000 interactions

Lambda	Alpha	Epsilon	Gamma
0.0000	0.3000	0.0050	0.6000

Figure 7

Quadratic regression models (univariate), with measured root mean squared error and coefficient of determination

**Table 7**

Quadratic regression curve vertex points on each hyperparameter at 100,000 interactions

Lambda	Alpha	Epsilon	Gamma
0.3729	0.3000	0.0050	0.7378

Looking at maxima and vertex points proposed by regression models from the start of the test (i.e. at 1000 interactions)²⁸, the proposed values for alpha and epsilon lie on the opposite ends of their respective value ranges compared to the ones proposed by models at end of the test. I extrapolate that the best performing configurations with little time to learn are those that adapt faster using high learning rates and high exploration rates, while over the long-term, the best-performing configurations are those that explore less and update their values more moderately.

The vertex points of epsilon and alpha shift substantially over the course of the test. For epsilon, the optimal value decreases from 0.0400 at 1,000 interactions to 0.0050 at 100,000 interactions. The tipping point occurs around 23,000 interactions, where the recorded vertex point is 0.0224. Alpha shows a similar pattern, falling from 0.5302 to 0.3000 over the test's duration. Lower values of alpha take over even sooner: by the 9,000th interaction, the vertex point of alpha falls below 0.4. In contrast, the vertexes of lambda and gamma shift by a far lesser degree throughout the test, with lambda decreasing from 0.4739 to 0.3729, and gamma increasing from 0.7030 to 0.7378.

The regression models presented in this chapter provide a basis for the subsequent analyses of hyperparameter behaviour throughout the AIQ test.

²⁸ The corresponding regression results are available in the supplementary repository (see [Appendix](#)) under `processed_Q-learning_EDA_results\regression`.

5 Q-Learning Performance Analysis Using Decision Trees

In the following analysis, I applied the `DecisionTreeRegressor` from the Scikit-learn library in order to identify which hyperparameter values lead to groups of configurations that obtain the highest AIQ scores at each recorded interaction step of the test. When deciding what depth of the tree model to use, I made sure that the resulting tree models would remain interpretable, as decision trees with an unbound range of parameters such as maximum depth or minimum sample split tend to overfit the data. In the most extreme scenario, a decision tree would fit a direct rule branch for all individual data points. To avoid this, I first constrained the models by using predefined maximum depth parameters, generating decision trees with a depth of 3-6. After evaluating the resulting models, I noticed some branches of the trees ended up reducing the node variance by very small amounts and did not meaningfully increase the model's informational value. Thus, I also used a minimum impurity decrease threshold (using the `min_impurity_decrease` parameter), which prevents the tree from creating a new split unless it reduces the node's MSE by at least a specified amount. A heuristically chosen value of 0.5 was used, set to a customizable variable declared at the start of the script. This threshold removes negligible splits, resulting in an increased readability of the resulting models while preserving the important branching rules within the tree models.

I created two sets of decision trees, one where only hyperparameters are considered as the independent variables, described in [Section 5.1](#), and one where the episode length is also taken into consideration with regards to the AIQ score, described in [Section 5.2](#).

5.1 Per-Episode Decision Trees

I generated sets of trees with varying depth for each recorded episode length of the AIQ test. The 100,000 interactions episode length trees for hyperparameters on AIQ are shown in Figures 8 and 9. As my goal within the EDA is not to create models capable of generalization, but rather to accurately describe the existing data, I did not use a train-test split to test the model's performance on unseen data, which resulted in noticeably low MSE and R^2 scores.

Figure 8

Decision tree regression model of depth 3 predicting AIQ on hyperparameters at 100,000 interactions

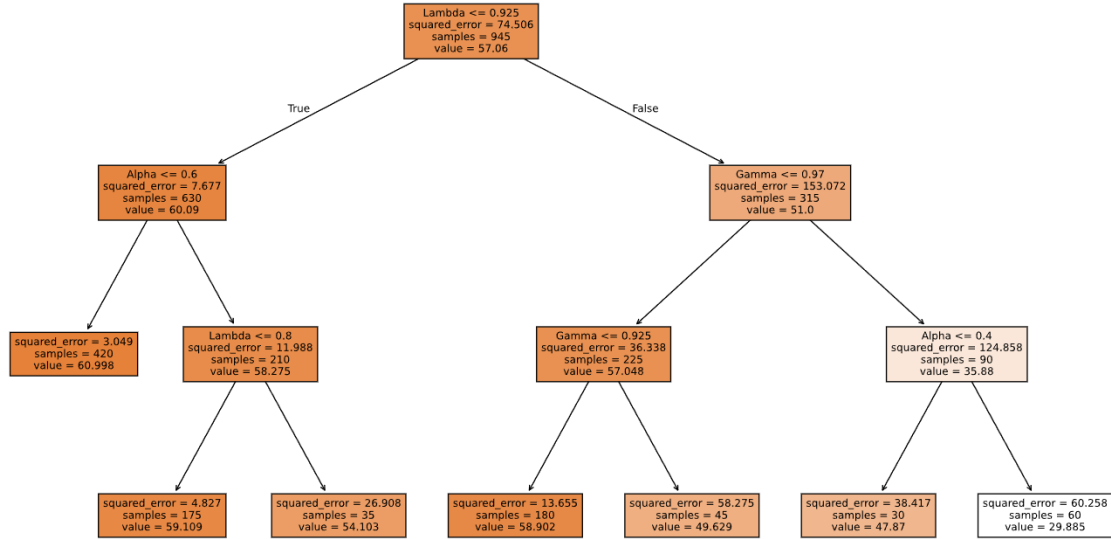
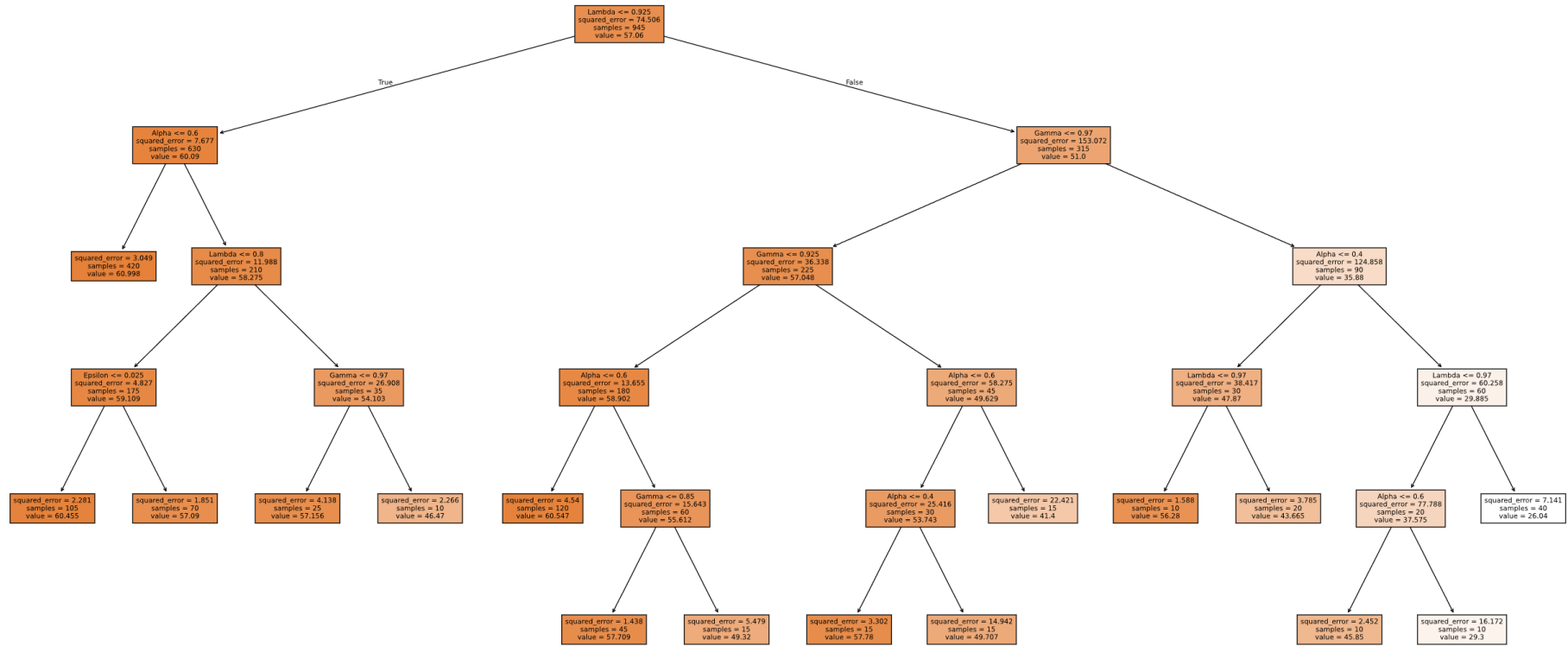


Figure 8 represents the simplest generated model within the analysis. While it does not show the more complex conditions of the deeper decision tree models, it highlights which hyperparameters have the strongest overall influence on Q-learning's performance at the end of the AIQ test. Notably, the upper extreme value of lambda is the primary criterion used for the first split, and none of the rules involve epsilon. The model also shows that the highest lambda values {0.95, 0.99, 0.995} may only work well in conjunction with the lower end of the tested gamma values {0.6, 0.7, 0.8, 0.9}. Many leaves show high variance (represented by the MSE), suggesting that deeper tree models may uncover splits that meaningfully reduce this variance.

Figure 9
Decision tree regression model of depth 5 predicting AIQ on hyperparameters at 100,000 interactions



As seen in Figure 9, the best leaf of the tree regression model of depth 5, with regards to the average AIQ value, is on the leftmost branch consisting of 420 configurations with an AIQ mean of 60.998, and an MSE of 3.049. Splits leading to this leaf rule out lambda values {0.95, 0.99, 0.995}, and alpha = 0.7. This aligns with the previous findings from [Chapter 4](#), where regression models showed a preference for lower eligibility trace decay rates and the lowest sampled learning rate at this stage of the AIQ test. Other leaves with comparably high AIQ scores suggest extremely high gamma is detrimental to good performance, again in line with previous findings.

Exploring the right side of the model, rules leading to the poorest performing configurations at the end of the AIQ test can be easily identified. The worst performing leaf consists of 40 configurations with extremely high gamma {0.99, 0.995}, lambda {0.99, 0.995} and moderate to high alpha {0.5, 0.7}, achieving a mean AIQ score of 26.04, with an MSE of 7.141. A smaller leaf of 10 configurations shows a slightly higher AIQ score of 29.3 with a substantially higher variance (MSE of 16.172). This subset shares the same extreme gamma values but uses lower lambda and exclusively the highest alpha value of 0.7. These leaves indicate that configurations with moderate to high learning rates, high eligibility trace decay rates and low discount to future rewards tend to perform poorly at an episode length of 100,000 interactions.

Next, I examine decision tree models from the early stages of the test. Figure 10 shows that at the start of the test, the best-performing configurations at the start of the AIQ test are characterised by moderate gamma values {0.6, 0.7, 0.8}, combined with epsilon {0.02, 0.03, 0.04}. This model is identical to the pruned model of depth 5 tree, meaning that no additional splits satisfied the minimal impurity difference condition. In other words, the model found no further partitions that would meaningfully decrease variance in the existing leaves.

Figure 10

Decision tree regression model of depth 4 predicting AIQ on hyperparameters at 1,000 interactions

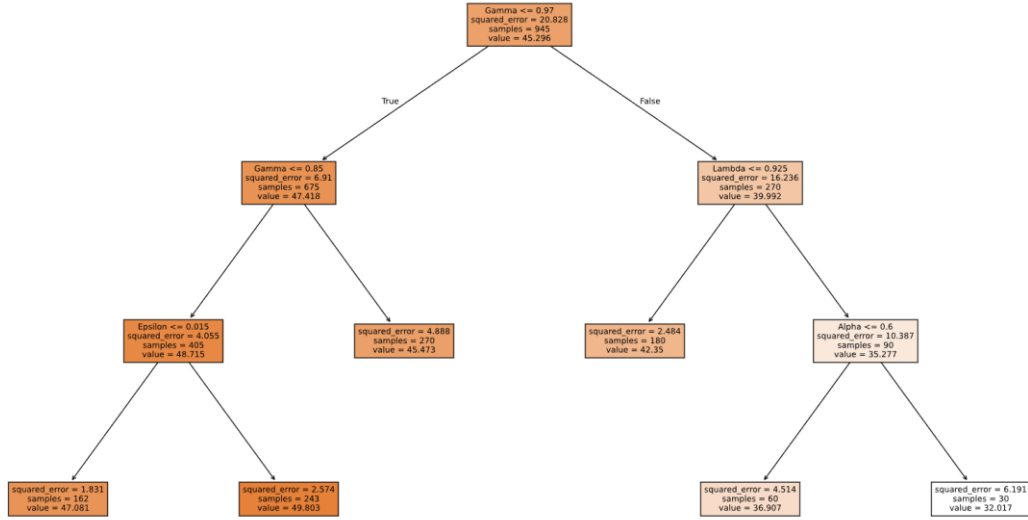
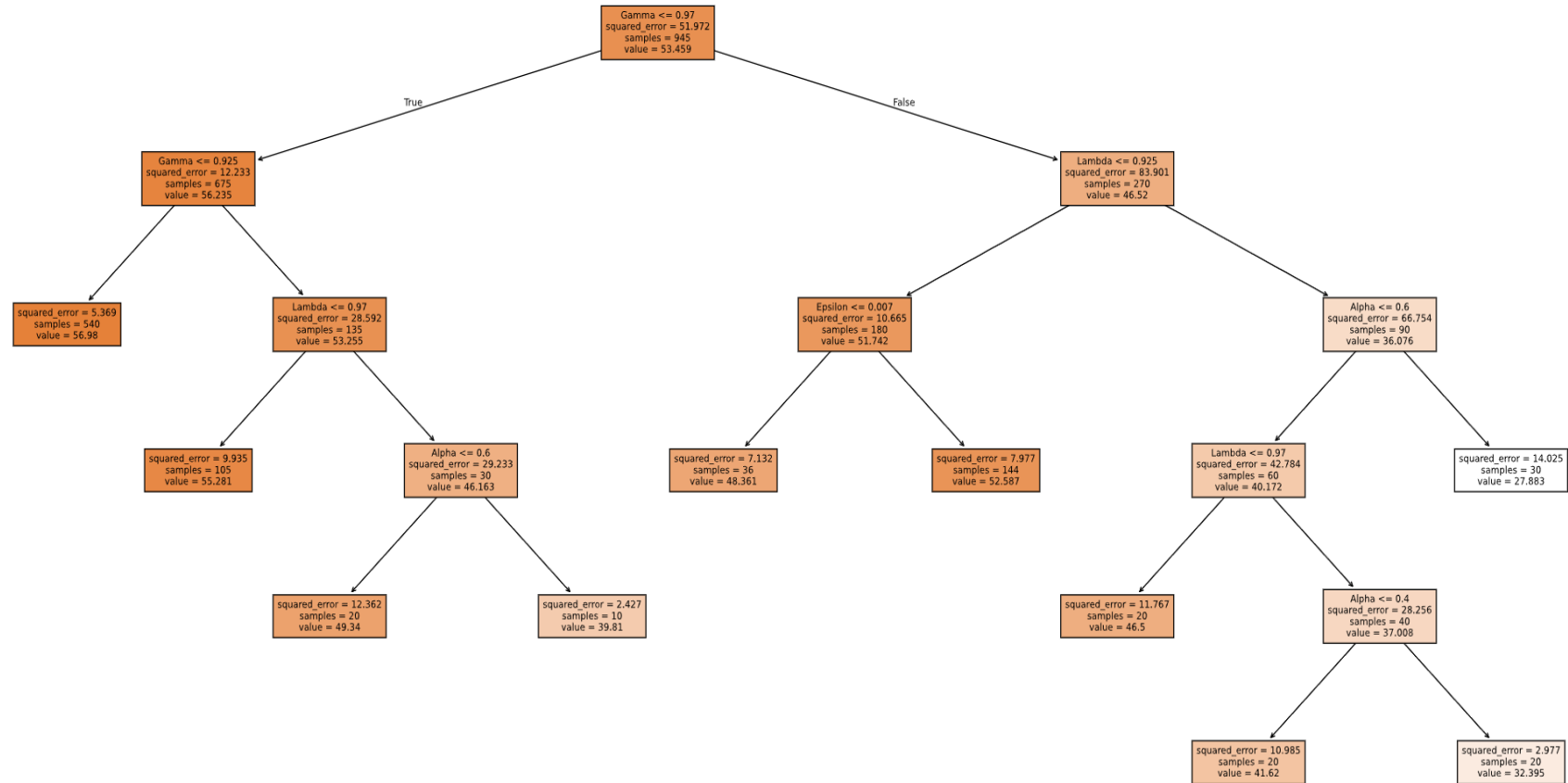


Figure 11 shows the depth 5 decision tree at 10,000 interactions, where the root split is determined by an upper extreme value of gamma. Configurations with $\text{gamma} \leq 0.9$ form the strongest leaf in the model, containing 540 of the 945 configurations, with a mean AIQ of 56.98. This model shows that high gamma and lambda values reduce AIQ performance, especially in conjunction with each other and high alpha.

Figure 11

Decision tree regression model of depth 5 predicting AIQ on hyperparameters at 10,000 interactions



5.2 Multi-episode Decision Tree

In this section, I examine a decision tree model that incorporates the episode length as an additional feature²⁹. Although four decision trees were generated³⁰, ranging in depth from three to six, I focus on the depth 6 model, which provides the highest degree of complexity, while remaining interpretable. Figure 12 shows the depth 6 model.

The depth 6 global decision tree again begins with the discount factor at the root node, indicating that an upper extreme value of gamma is the strongest determinant of AIQ performance, even with episode length included as a feature.

The episode length appears only once in the left branch of the tree: configurations with fewer than 9,000 interactions form a tree with a mean AIQ of 52.769 and a high MSE of 18.965, while configurations with a longer time to learn benefit from lower learning rates of $\alpha \leq 0.5$, yielding a leaf with a mean AIQ of 59.856, a lower but still considerably high MSE of 8.05, containing 41400 records (where the same configurations may keep reappearing, as opposed to the episode-specific decision trees, where each leaf sample represented one unique configuration).

The right side of the tree is dominated by splits containing lambda and alpha, with episode length again appearing only once, distinguishing weaker performers at $EL \leq 14,000$ interactions. Configurations with more time to learn benefit from $\lambda \leq 0.8$, and $\gamma \leq 0.95$.

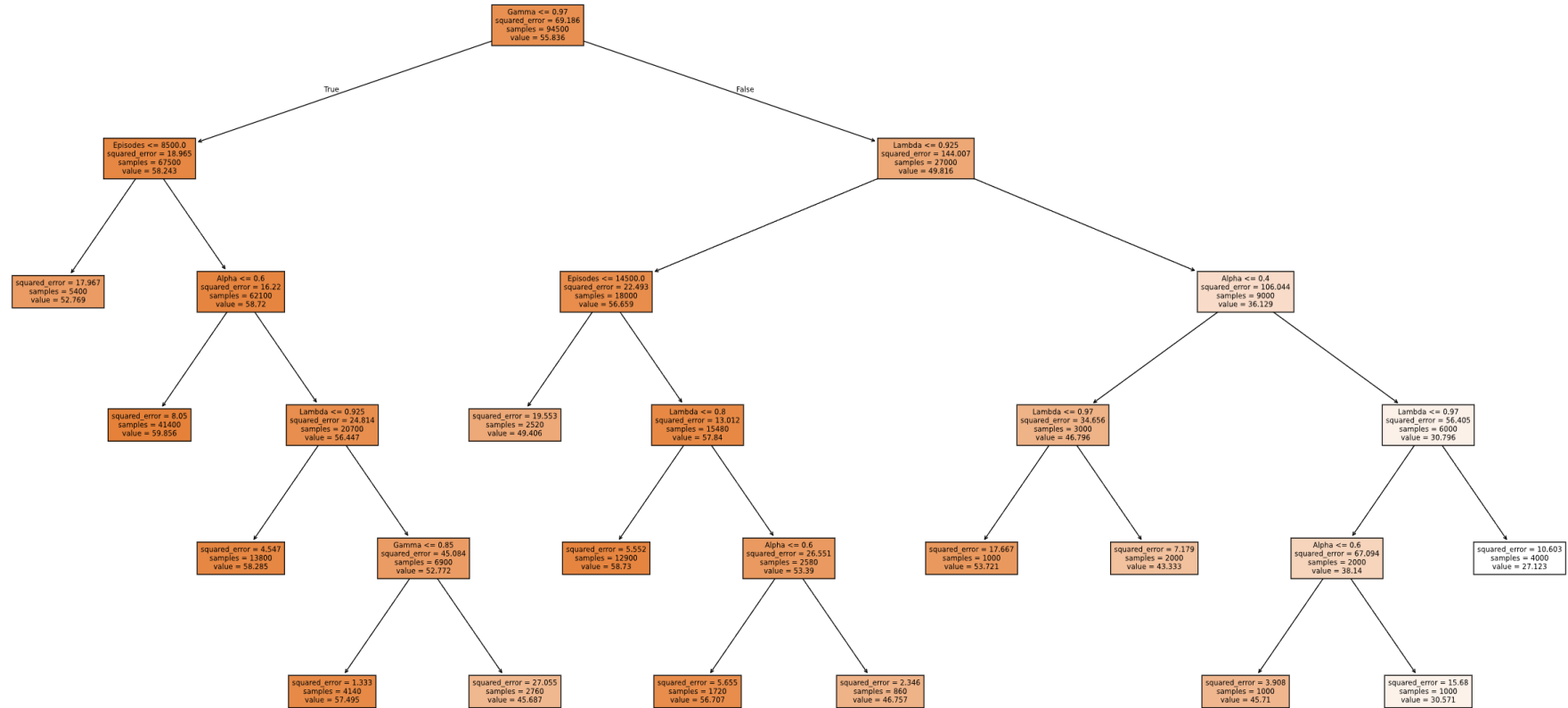
Overall, the decision tree models show that interactions among gamma and lambda are the main factors in Q-learning performance, with episode length, alpha and epsilon exerting comparatively lesser influence on the AIQ score.

²⁹ This is why, as opposed to trees generated for each recorded episode length, I refer to this model as global.

³⁰ Lower depth trees may prove useful for other AIQ test experiments and are therefore kept within the script.

Figure 12

Decision tree regression model of depth 6 predicting AIQ on hyperparameters and episode length.



All decision tree models, including those generated without the minimal impurity difference threshold are provided within the supplementary repository under `data/processed_Q-learning_EDA_results_updated/decision_trees` (see [Appendix](#) for details on directory structure and naming conventions).

6 Q-Learning Performance Analysis Using Time Series

As the data in the full dataset corresponds to a time series, I generated a series of line charts to visualize it. As plotting all 945 configurations at once would make the resulting graphs hard to interpret, I decided to filter the configurations, focusing only on the high-performing Q-learning configurations at a given interaction episode. [Section 6.1](#) describes the filtering method used to extract the indistinguishably best-performing configurations, and the normalisation procedure for plotting normalised time series plots. [Section 6.2](#) identifies configurations that exhibited consistent high performance throughout the AIQ test. Finally, in [Section 6.3](#) I examine three configuration types characterised by the shape of their AIQ score curves and the underlying hyperparameter configuration trends.

6.1 Confidence Interval Based Filtering and AIQ Normalization

Throughout the analysis, I used two approaches to filter configurations: filtering using the confidence interval overlap, and percentile-based filtering, that is used in [Chapter 8](#). For the time series analysis, I devised a simple filtering algorithm to filter for the indistinguishably best configurations at all recorded steps of the AIQ test, based on the 95% confidence interval overlap between the highest scoring agent and other configurations. In other words, if the 95% confidence interval of a configuration's AIQ score does not overlap with the 95% confidence interval of the best-performing configuration, then their scores are significantly different and the former configuration is filtered out. Conversely, configurations whose confidence intervals do overlap with the best-performing configuration cannot be ruled out as potentially equivalent and are retained for the time-series analysis. To visualise the boundary by which configurations are being filtered, I added a dotted line to all plots representing the lower confidence interval bound of the best performer at each episode.

To implement this filtering method, I calculated the upper bound of the AIQ confidence interval. After identifying the best agent using this value, named `CI_upper` within the script, I used the other side of its confidence interval (`CI_lower`) as a threshold: any configuration whose upper confidence interval is greater than or equal to this value is considered statistically indistinguishable from the best performer at that episode. I then plotted these

configurations on a time series graph, showing their overall performance throughout the AIQ test with both standard AIQ values and their normalised values, another derived variable.

To normalize the AIQ scores, I opted for per-episode min-max normalization, setting the highest AIQ recorded at each step as 1, with the lowest AIQ at each step set as the bottom of the normalized value. This value is represented in code as `AIQ_norm`. Formally, the normalised value `AIQ_norm` at episode e is computed as

$$AIQ_norm(e) = \frac{AIQ(e) - \min(AIQ_e)}{\max(AIQ_e) - \min(AIQ_e)}, \quad (4)$$

where $\min(AIQ_e)$ and $\max(AIQ_e)$ denote the minimum and maximum AIQ values at that episode.

6.2 Top Q-learning Configurations by Frequency

The filtering and normalization approach described in [Section 6.1](#) allowed me to visualise the performance of all filtered configurations throughout the AIQ test, and to search for configurations that were indistinguishably best at the start of the test and maintained good performance throughout the 100 recorded time series points of the AIQ test. To achieve this, I recorded the frequency with which each configuration passed the filtering threshold and exported the recurring configurations as a `.csv` file along with the count of times they passed the threshold, in a descending order. The resulting file is further filtered by a customisable frequency percentile variable, ensuring only the most often recurring configurations are included. Table 8 shows the 90th percentile of these recurring configurations, subset for their hyperparameter values and the frequency column.

Table 8
Top configurations in the 90th-percentile frequency subset, sorted by frequency

Lambda	Alpha	Epsilon	Gamma	Frequency
0.5	0.5	0.01	0.8	94
0.7	0.3	0.02	0.8	92
0.5	0.3	0.02	0.8	90
0.5	0.5	0.01	0.9	87

0.9	0.3	0.01	0.6	87
0.3	0.5	0.01	0.9	85
0.7	0.3	0.01	0.7	85
0.7	0.3	0.02	0.9	84
0.5	0.5	0.01	0.95	84
0.995	0.3	0.01	0.6	83
0.9	0.3	0.01	0.7	83
0.995	0.3	0.01	0.7	83
0.5	0.3	0.01	0.7	82
0.7	0.3	0.01	0.6	82
0.99	0.3	0.01	0.7	82
0.95	0.3	0.01	0.7	80
0.99	0.3	0.01	0.6	79
0.5	0.3	0.01	0.8	79
0.7	0.3	0.01	0.8	79

Note. Source file: `learning_indist_top_performers_90th_percentile_by_freq.csv`, located in `data/processed_Q-learning_EDA_results/time_series/configs`. The full repository structure is listed in the [Appendix](#).

Among these top-performing configurations, lambda values range from 0.3 to 0.7, or cluster near 1 when paired with especially low alpha (and to a lesser extent gamma). A common characteristic is a low exploration rate either at or below 2%. I found the number of configurations with high eligibility trace decay rates in the sample surprising, as the lambda line chart with confidence intervals suggested poorly performing configurations at these values. While multiple types of configurations show good performance throughout the test, several tendencies are visible: high gamma values are mostly absent, epsilon is always at or below 2%, alpha of 0.7 does not appear, and there is a noticeable inverse tendency between

lambda and alpha values, with higher learning rates typically paired with lower eligibility trace decay rates and vice versa.

6.3 Patterns in Agent Performance

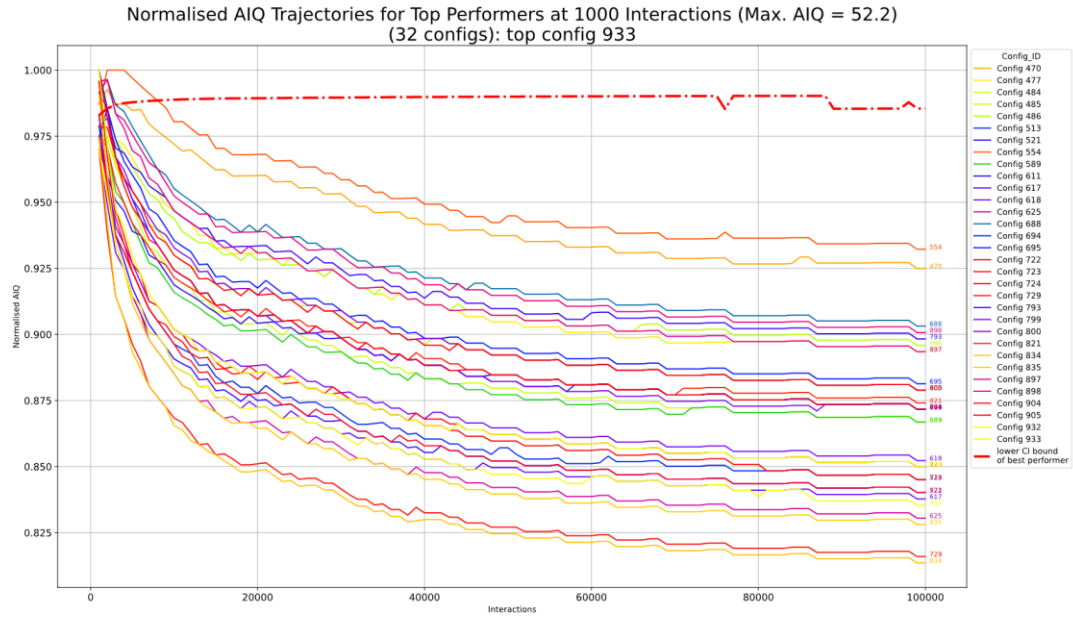
Next, I examined the normalised time series plots, which I deemed superior in terms of informational value compared to the raw (non-normalised) time series plots, as they allow for comparing agent performance against the highest performing agent at each interaction episode, despite that agent not being depicted. I identified three main types of configurations with regard to the shape of their performance curves, each becoming prevalent at different stages of the AIQ test: short-term best performers, shown in Figure 13, intermediate best performers, shown in Figure 14, and long-term best performers, shown in Figure 15. The names of these categories refer to configurations that tend to become indistinguishably best during the early, intermediate, or late phases of the AIQ test. When examining all normalised time series plots, it was difficult to pinpoint exactly where each of the three categories start appearing, discounting of course the short-term best performers, as the transition seems smooth and gradual. By 15,000 interactions, the majority of recorded configurations remained in the 95th AIQ percentile throughout the test (intermediate high-performers), and at around 60,000 interactions, a sizeable number of recorded configurations displayed an initial drop in the normalised AIQ score (long-term high-performers).

6.3.1 Highly-performing Configurations at the Start of the AIQ Test

I will now look at interesting configurations at the start of the AIQ test. Figure 13 shows that despite the fact that 28 out of 32 configurations which passed the confidence interval overlap threshold fell below the 90th percentile, Configurations listed in Table 9 remained fairly competitive, maintaining their AIQ scores within the 90th percentile. Table 9 shows the hyperparameters of these configurations.

Figure 13

Time series graph of top performers at 1,000 interactions throughout the AIQ test with normalised AIQ score.

**Table 9**

AIQ and hyperparameter data of top performers at 1,000 interactions which remained in the 90th percentile at the end of the test.

Config. ID	EL = 1,000 interactions		EL = 100,000 interactions		Hyperparameters			
	AIQ	HCI	AIQ ³¹	HCI	Lambda	Alpha	Epsilon	Gamma
554	51.9	0.5	61.1	0.4	0.9	0.3	0.04	0.6
470	51.9	0.4	60.8	0.4	0.7	0.5	0.02	0.6
688	52.1	0.5	59.9	0.4	0.95	0.5	0.03	0.7
898	52.0	0.4	59.8	0.4	0.995	0.5	0.03	0.7

³¹ The maximum AIQ from the whole dataset at this episode length is 63.9, with a HCI of 0.4.

Examining the configurations that remained within the 90th percentile at 100,000 interactions, I found several shared characteristics. These configurations use medium-high exploration rates of 2-4%, which is mostly consistent with the regression prediction for well-performing configurations at the start of the test, established in [Chapter 4](#). Conversely, three of the configurations employ high eligibility traces ($\lambda \geq 0.9$), whereas quadratic regression predicted a value of 0.4739 at the early stages. Regression models also suggested that low learning rates are detrimental to scoring high AIQ scores in the short term, yet the first configuration listed in Table 9 has the lowest discrete alpha value of 0.3. This does however align with the fact that regression indicated this learning rate value as optimal for the final stages of the AIQ test, as this configuration is the strongest long-term performer in the subset.

Other top performers shown in Figure 13, representing the previously established short-term best performers, use higher exploration rates of 3-4%, learning rates of 0.5-0.7, discount factors between 0.7-0.8 and eligibility trace decay rates of at least 0.7. These value ranges describe configurations that update their Q-values quickly and attribute credit relatively far back using their eligibility traces, while exploring quite often. This corresponds with their ability to quickly achieve high AIQ scores, however the long-term AIQ of these configurations declines due to the higher exploration rates, which limit the maximum achievable average reward in later stages of the test.

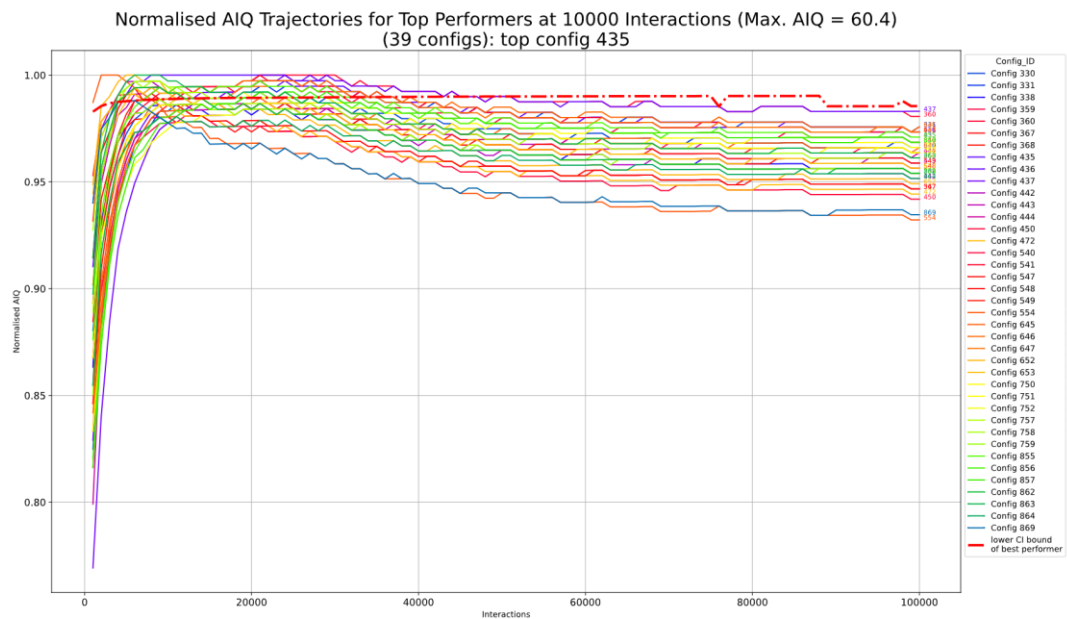
6.3.2 Highly-performing Configurations at Intermediate Stages of the AIQ Test

As mentioned earlier, the shape of the performance curve depicted by the normalized AIQ score generally varies when subsetting configurations that satisfied the confidence interval overlap condition at different stages of the test. This can be seen when comparing Figures 13, 14 and 15. Figure 14 shows a time series graph displaying configurations that were indistinguishably best-performing at 10,000 interactions. Whereas the comparative performance of most configurations passing the CI bound threshold at 1,000 interactions decreased quickly as the test progressed, configurations shown in Figure 14 display a different pattern: many performed relatively poorly at the test's onset, but quickly achieved high AIQ scores, after which their performance declined much more slowly compared to the previous configuration subset. As this shape of the performance curve is most prominent within the early-middle stages of the test, roughly between 6,000 and 50,000 interactions, I decided to refer to these as medium-term best performers.

Inspecting the best performers at 10,000 interactions, I noticed they typically used exploration rates of 2-3% (with three exceptions from a set of 39), only low to mid-range learning rates (most commonly 0.3), lower discount factors between 0.6 and 0.8, and eligibility trace decay rates of at least 0.5. This describes configurations that learn at a modest rate, focusing on comparatively short rewards with a moderate degree of exploration.

Figure 14

Time series graph of top performers at 10,000 interactions throughout the AIQ test with normalised AIQ score.



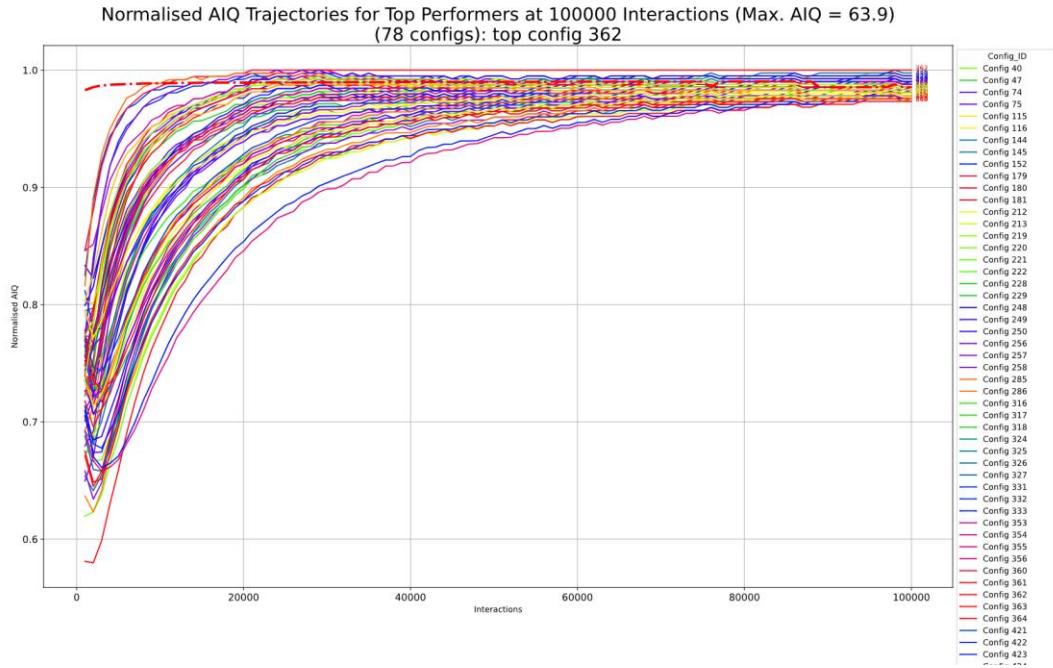
6.3.3 Highly-performing Configurations at the Final Stages of the AIQ Test

Lastly, the long-term best performers, shown in Figure 15, are characterized by an initial drop in normalized AIQ, as the two previously mentioned groups begin learning quicker. As time progresses, these configurations begin to catch up and take over. As can be seen in Figure 15, all configurations that passed the confidence interval overlap at the last recorded step of the AIQ test have entered the 90th percentile of performers by 35,000 interactions. These configurations employ exploration rates of 0.5-1%, learning rate of 0.3, discount factors ranging from 0.6, with the higher end of above 0.9 being prevalent, and eligibility trace decay rates across the entire range of their value space. Therefore, unlike the short and medium-term best performers, these configurations are the only ones that seek long-term

rewards. These long-term best performers rarely explore and update their Q-values as slowly as possible given the available discrete learning rates in the dataset. Interestingly, some of the configurations do not use eligibility traces whatsoever, which seems to be compensated by a high discount factor³².

Figure 15

Time series graph of top performers at 100,000 interactions throughout the AIQ test with normalised AIQ score



In conclusion, the efforts undertaken to perform the time series analysis resulted in a useful tool for gathering insights about various configurations and configuration types throughout the AIQ test. Although the large number of filtered configurations tends to reduce the readability of the resulting plots, in particular when identifying which line corresponds to which configuration, the plots are complemented by .csv files listing filtered configurations for each episode, which complement the plots and and mitigate the issue. The plots and tables of configurations belonging to the listed categories are provided within the supplementary repository under `data/processed_Q-learning_EDA_results_updated/time_series` (see [Appendix](#) for directory structure and file naming).

³² The four configurations that have their eligibility trace decay rates set to zero have discount factor values of either 0.9 or 0.95.

7 Attempting Clustering Analysis to Categorise Q-Learning Configurations

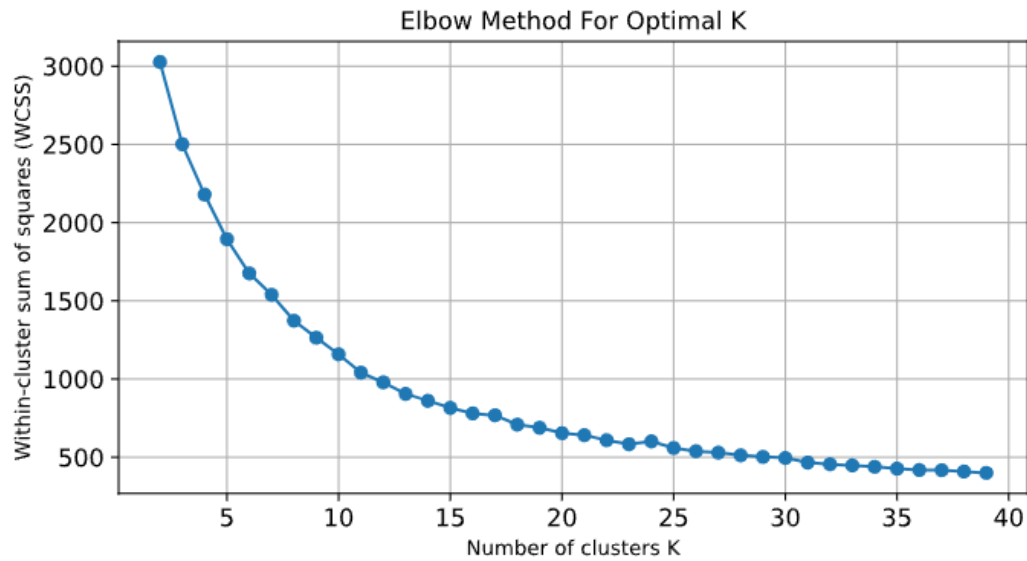
This chapter describes my undertakings when employing clustering analysis on the provided data. In [Section 2.3.3](#), I pre-established that the silhouette score is a valuable, mathematically grounded metric for the identification of optimal cluster count, K . I also hinted that it would be the preferable metric for me to base the analysis on, in comparison to the heuristic elbow method. Ultimately, however, employing either one of the methods proved difficult. After employing the silhouette score in an if-else loop of the code devised to identify optimal K , I found that the implemented condition triggered at $K=3$: despite marginal improvements in the silhouette score with each additional cluster, between $K=3$ and $K=4$, the score worsened by a value of 0.00085, resulting in the code designating 3 as the optimal number of clusters. This was a problem encountered only with the addition of new configurations (see [Section 3.2.3](#)), as before, this method designated 7 as the optimal K , which I deemed satisfactory at the time³³. While it would be possible to modify the selection rule to tolerate such marginal decrease in the silhouette score, the silhouette scores remain low across all computed K values, ranging from 0.20364 at $K=2$ to 0.31870 at $K=39$. Whilst trying to employ the elbow method to circumvent the issue, I found that, just as the silhouette score increases very gradually, the WCSS³⁴ decreases in a similar manner. Figure 11 shows the plot of WCSS on K , highlighting the difficulty encountered when attempting to identify the “elbow”.

³³ Although my findings at the time also concluded clustering’s unsuitability for the data.

³⁴ Within-cluster sum of squares, a metric used in the elbow method to identify the “elbow”, i.e., the point where optimal K is found using this method.

Figure 11

Plot of within-cluster sum of squares on number of clusters K



This has led me to explore the validity of K-means in my use-case. As Géron (2022, Chapter 9, Limits of K-means) states: “k-means does not behave very well when the clusters have varying sizes, different densities, or nonspherical shapes.” This, along with the observed metrics and multidimensional character of the hyperparameters’ effects on AIQ, led me to conclude that K-means is ultimately not an insightful method for this analysis.

8 Association Rule Mining Using CleverMiner

This chapter applies association rule mining to identify combinations of hyperparameter values that are associated with high-performing Q-learning configurations. I used CleverMiner’s 4ft-miner procedure to extract association rules on two different top-performer selection criteria, categorising the continuous AIQ variable. The first approach utilised the indistinguishable top performer logic, consisting of the confidence interval overlap, described in [Chapter 6](#), categorising the AIQ score into the ‘Indistinguishable_Best’ and ‘Lower’ classes. The second, percentile-based categorisation is used as a supplementary perspective: it allows for a wider range of hyperparameter values to pass the positive target class requirements, thereby allowing rules with higher base (support) to be discovered at the cost of including configurations with somewhat weaker AIQ scores.

Association rules were mined using a variable minimum support base and confidence threshold for each 1,000-interaction step, and hyperparameters were combined using logical conjunction. All rules satisfying the base and confidence thresholds were written to an episode-specific output file, containing 4ft-Miner’s built-in evaluation metrics (Masa & Little Big Company, 2025). The output is easily readable, and the base and confidence thresholds can be modified in the analysis variable set-up section. I will now go over both association rule mining approaches and present a selection of rules that were found.

[Section 8.1](#) introduces rules found with the confidence overlap discretisation rule. [Section 8.2](#) describes rules found using the percentile-based discretisation rule.

8.1 Mining Association Rules Using the Confidence overlap

The first application of 4ft-miner association rule mining uses the per-episode confidence overlap filtering technique from [Chapter 6](#). 4ft-miner identifies rules consisting of complementary hyperparameter values which result in Q-learning achieving indistinguishably best AIQ scores at each interaction step. Using a minimum base 5 and confidence of 80% I was able to find a large number of rules, often with partial overlap. The

base of most of the rules found was either 7 or 8, however after 97,000 interactions, two rules with a base of 13 were found, as presented in Table 10.

Table 8

2 selected association rules with the highest base discovered at 97,000-100,000 with the Indistinguishable_Best positive AIQ class

Base	Conf.	AAD ³⁵	Lambda	Alpha	Epsilon	Gamma
13	0.812	11.587	0.3 0.5	0.3 0.5	0.005 0.01	0.8 0.9
13	0.812	11.587	0.3 0.5	0.3 0.5	0.005 0.01	0.8 0.95

As evident, there is a large overlap between these two rules, with the only difference between them being the inclusion of a different discount factor value (0.9 and 0.95). These rules support previous findings about high-performing configurations at the end of the test, in that low learning and exploration rates play key roles when Q-learning has enough time to learn about the environment. Compare these findings to the rules found at the start of the test, shown in Table 11.

Table 11

All association rules discovered at 1,000 interactions with the Indistinguishable_best positive AIQ class

Base	Conf.	AAD	Lambda	Alpha	Epsilon	Gamma
7	0.875	24.840	0.7 0.995	0.5	0.03 0.04	0.6 0.7
7	0.875	24.840	0.95 0.99	0.5 0.7	0.04	0.6 0.7
7	0.875	24.840	0.95 0.995	0.5	0.03 0.04	0.6 0.7
7	0.875	24.840	0.95 0.995	0.5 0.7	0.03	0.6 0.7
7	0.875	11.587	0.99 0.995	0.5	0.03 0.04	0.6 0.7

There is once again a large overlap between the found rules. The hyperparameter values discovered within these rules corroborate the findings in the previous analyses: at the start of the AIQ test, highly performing configurations display high learning and exploration rates and employ high eligibility trace decay rates, with the discount factor set to either 0.6 or 0.7.

³⁵ Above Average Difference, $P(S|A)/P(S)-1$. It is a measure of how much the antecedent improves the probability of S compared to all records (Masa & Little Big Company, 2025).

8.2 Mining Association Rules Using Percentile-Based Categorization

For the second part of the association rule mining analysis, I opted for percentile-based categorization, using a customizable list variable `cleverminer_percentiles` to allow for varying categorization of the AIQ scores. Overall, I performed categorisation of the AIQ score using the 85th, 90th, and 95th percentiles. Association rule mining was set for bases of 5, 10, and 20, using a minimum confidence threshold of 80%. To contrast the relatively low base of the rules found in Section 8.1, I will focus on the strongest base and confidence rules found at 1,000, 10,000 and 100,000 interactions using a minimum base of 20 on the 85th percentile split of the AIQ score.

At 1,000 interactions, the 85th-percentile threshold corresponds to an AIQ score of 49.8 (highest AIQ = 52.2 at this EL). 63 rules for the `Top_85th_Percentile` AIQ class were found, with rules combining high eligibility trace decay rates, relatively high exploration rates and moderate discount factors. Table 12 shows two rules, first displaying 100% confidence and the second having the highest base out of all rules found in this selection.

Table 12

2 selected association rules discovered at 1,000 interactions with the `Top_85th_Percentile` positive AIQ class

Base	Conf.	AAD ³⁶	Lambda	Alpha	Epsilon	Gamma
24	1.000	5.342	0.95 0.995	-	0.030 0.040	0.6 0.7
59	0.819	4.197	-	0.5 0.7	0.030 0.040	0.6 0.7

Next, at 10,000 interactions, the 85th AIQ percentile threshold rises to 58.5 (recorded maximum being 60.4). Only two rules satisfying the conditions were discovered at this stage. These rules, depicted in Table 13, show significant overlap. They involve moderately low exploration rates and mid-range learning rates, indicating a shift from the more exploratory behaviour observed at 1,000 interactions.

³⁶ Above Average Difference, $P(S|A)/P(S)-1$. It is a measure of how much the antecedent improves the probability of S compared to all records (Masa & Little Big Company, 2025).

Table 13

2 association rules discovered at 10,000 interactions with the Top_85th_Percentile positive AIQ class

Base	Conf.	AAD ³⁷	Lambda	Alpha	Epsilon	Gamma
29	0.806	3.849	-	0.3 0.5	0.020	0.6 0.8
29	0.806	3.849	-	0.3 0.5	0.020	0.7 0.8

At the final episode length of 100,000 interactions, the 85th-percentile threshold further increases to an AIQ score of 62.34. The six discovered rules, once more with significant overlap, are shown in Table 14. These rules feature very low learning and exploration rates, and lower range of gamma values, with a notable exception of the first rule, where the high gamma value is compensated by low lambda.

Table 13

2 association rules discovered at 10,000 interactions with the Top_85th_Percentile positive AIQ class

Base	Conf.	AAD	Lambda	Alpha	Epsilon	Gamma
20	0.833	4.546	0.1 0.3	-	0.005 0.010	0.9 0.95
30	0.833	4.546	-	0.3	0.005 0.010	0.6 0.7
30	0.833	4.546	-	0.3	0.005 0.010	0.6 0.8
32	0.889	4.915	-	0.3	0.005 0.010	0.7 0.8
29	0.806	4.361	-	0.3	0.010 0.020	0.6 0.8
31	0.861	4.731	-	0.3	0.010 0.020	0.7 0.8

All rule lists are provided within the supplementary repository under `data/processed_Q-learning_EDA_results_updated/association_rules` (see [Appendix](#) for details on directory structure and naming conventions).

³⁷ Above Average Difference, $P(S|A)/P(S)-1$. It is a measure of how much the antecedent improves the probability of S compared to all records (Masa & Little Big Company, 2025).

9 Discussion

In this chapter, I interpret the results of the exploratory analyses presented in [Chapters 4-8](#). I first summarise the main patterns uncovered using the employed methods, before discussing each method's contribution in greater detail. Finally, I reflect on the methodological limitations and outline implications and avenues for further research on Q-learning hyperparameter tuning within the AIQ test framework, along with remarks on possible improvements to the python script used to conduct the EDA.

9.1 Cross-Method Synthesis

Combining the results of the four analytical methods employed within the thesis offers a picture of how the Q-learning hyperparameters shape agent performance throughout the AIQ test. Regression, time series analysis and association rule mining all indicate that the learning rate and exploration rate play a substantial role in determining how quickly agents improve at various stages of the test, with a high value combination of these values leading to potentially good performance in the short-run, and low values benefiting agents in the long-run. These findings show that a high degree of exploration and learning allows Q-learning to quickly learn about the environment, but prevent it from accumulating high rewards at longer episode lengths. The conjunction of similar value ranges between these two hyperparameters seem to form a basis for good performance when accompanied by a fitting non-extreme pairing of gamma and lambda, i.e., a very high value of one needs to be compensated by a moderate value of the other³⁸, representing an interesting trade-off. A high value of lambda, combined with high alpha and epsilon seems to favor short-term best-performers, with the importance of the high eligibility trace decay rates decreasing throughout the AIQ test.

Decision trees complement these findings by looking at broader influences of various hyperparameters, revealing which hyperparameters explain the largest performance

³⁸ The combination of extreme gamma and lambda points to poor performance, as observed in [Chapter 5](#).

differences across all 945 configurations. In this context, gamma and lambda consistently appear at the highest-level splits, splitting the samples at the upper extremes of their value ranges (typically above 0.9). These splits separate configurations that use very strong discounting or eligibility trace decay rates. However, as mentioned above, successful configurations were found that compensate one extreme upper value with a moderate value of the other.

Together, these findings outline Q-learning behaviour across the duration of the AIQ test. I will now discuss each of the analyses' individually to offer a more detailed view of their specific contributions and insights.

9.2 Regression analysis

The univariate regression models of Q-learning's hyperparameters on the AIQ score (Figures 6 and 7) offer an insight into how optimal configuration values shift throughout the AIQ test. In particular, the vertex points of the quadratic models, which performed better than linear models with regards to the employed performance metrics, highlight how the preferred values of individual parameters evolve with increasing episode length.

These regression patterns suggest a direction for further experimentation with regards to configuration sampling. Models of epsilon at early stages of the test indicate that even higher exploration rates may be worth employing when performing future AIQ tests. Similarly, at the end of the test, models put both epsilon and alpha at the bottom of the tested range, suggesting that values below those included in the Q-learning configurations could yield better long-term results.

9.3 Decision Tree Analysis

The decision tree analysis provides an interpretable view of how Q-learning hyperparameters influence performance at different stages of the AIQ test. Across all models, the trees consistently place the discount factor and eligibility trace decay rates at the highest levels, indicating that gamma and lambda possess the greatest influence on Q-learning's performance, while the effects episode length, alpha, and epsilon tend to appear only in lower-level splits. In several cases, these patterns contrasted with my earlier expectations based on regression models and descriptive statistics: I anticipated that the learning rate would play a more prominent role in the early stages of the test, yet the models

for the beginning stages of the test (Figures 10 and 11) relied far more heavily on gamma, lambda, and epsilon.

The worst-performing leaves examined in the depth 5 tree model (Figure 9), aligning with the information gathered from histograms (Figure 3) offer further insight into configurations with worsening performance through the AIQ test. Configurations with a combination of high gamma and lambda repeatedly fell into the lowest-scoring nodes, especially when combined with high alpha. These configurations likely suffer from unstable Q-value updates that accumulate over time. Conversely, leaves associated with the highest AIQ scores almost always exclude upper extremes of gamma and lambda, favouring moderate discount rates and smaller eligibility trace decay rates.

The global decision tree (Figure 12) introduces episode length as an additional feature. I expected the decision tree would leverage this new feature to a greater extent, yet it only selected the episode length as a splitting criterion twice. Instead, the model continued to heavily rely on gamma and lambda, with episode length displaying moderate influence, although still greater than alpha and epsilon. These findings suggest that episode length is overshadowed by stronger, more consistent relationships between gamma and lambda.

9.4 Time Series Analysis

The time series analysis shows that Q-learning configurations do not follow a single performance trajectory: within the analysis, I identified three main characteristic learning patterns that describe different configuration types and explored each category's distinct hyperparameter ranges. The observed hyperparameter ranges in each category could serve as a starting point for sampling a narrowly focused set of configurations that are likely to perform well within various learning time frames. Filtering configurations using the 95% confidence interval overlap proved to be a useful way to identify long-term high-performing configurations and examine their characteristics.

9.5 Association Rule Mining Analysis

The association rule mining results complement the previous analyses by highlighting groups of hyperparameter values that are prevalent among high-performing configurations at various stages of the AIQ test. The discovered rules do not uncover any new trends with

regard to what makes a well-performing configuration but rather reaffirm what I have discovered in the previous analyses.

The high degree of overlap among rules within each episode highlights the presence of narrow hyperparameter value ranges resulting in good performance at various stages of the AIQ test. For example, the rules found at the end of the test showed that low learning and exploration rates are seemingly a necessity for achieving the highest AIQ scores, typically in combination with moderate discount factors and eligibility trace decay rates.

9.6 Reflections on Methodology

The exploratory data analysis conducted within this thesis consists of a range of methods with varying degrees of suitability and interpretability for the examined data. While each successfully employed method helped form a clearer perspective on hyperparameter behaviour in Q-learning, several methodological limitations became apparent. The most notable limitation I encountered stems from the simplicity of the used regression models. Although the univariate regression models capture the broad relationships between hyperparameters and the AIQ score, they do not fit the observed data particularly well. The models outline general trends in hyperparameters' effects on AIQ score, but they do not uncover complex interactions between hyperparameters in high-performing configurations.

Another limitation was encountered when generating the time series plots of configurations gathered using the confidence interval overlap filtration technique. At various stages of the test, the high number of configurations that passed the filter resulted in performance curve overlap, legends not fitting in the charts and insufficient colour distinction of configurations, hindering the interpretability of the plots. I mitigated this problem by generating files containing recorded configurations for each recorded episode length, but it is a work-around that does not satisfyingly resolve the issue.

Finally, my limited experience with advanced data analytics, combined with the multidimensional character of hyperparameter relationships in Q-learning, means that the analysis relies on several individual, relatively simple methods. No single model, or indeed the resulting python script, presents a unified, simply presentable answer to the problem of optimal configuration identification at a given episode length. Nonetheless, the combination of methods forms a meaningful and multi-angled insight into Q-learning performance.

9.7 Implications and Further Work

The employed regression models, showing marginal improvement in the employed evaluation metrics between linear and quadratic regression, suggest that multivariate quadratic, or a higher degree regression method could produce much better models – although such types of regression were attempted, I ultimately failed to successfully implement them.

The challenges encountered in the time series visualisation point toward interactive plotting tools, either using more advanced python libraries or external software, as an elegant solution potentially solving the issue, allowing for interactive filtering and resulting in improved interpretability.

Lastly, there are numerous opportunities for improving the Python script as a whole. These include optimisation with regards to used data types and looping logic, or improvements to the set-up stage, making it more user-friendly. A very interesting direction would be designing logic to collect notable findings in a summary style, or to automatically identify promising hyperparameter intervals, sorted into distinct configuration types. Such functionality could be then built into a recommendation system for promising configurations at a given episode length.

Conclusions

I set several goals for this thesis, both theoretical and practical. The first goal was to use relevant literature to introduce key concepts necessary to understand the evaluation of artificial agents, with emphasis on the AIQ test and the reinforcement learning algorithm examined in this work, Q-learning. I consider this goal achieved, and the work can partly serve as an introduction to these topics.

A second goal was to conduct an explorative data analysis to examine how combinations of Q-learning hyperparameters influence agent performance across the duration of the AIQ test. I identified patterns in short-term and long-term performance using regression models, decision trees, time series analysis and association rule mining. Namely, the results show that learning and exploration rates influence the speed at which agents improve, but high values of these hyperparameters hinder performance at higher episode lengths. A combination of high discount factors and eligibility trace decay rates are especially detrimental to agent performance, but a moderate value of one may compensate the other, and moderate alpha can similarly compensate high eligibility decay rates. The findings also point to unsampled value ranges of alpha and epsilon as potentially resulting in better performers at certain stages of the AIQ test: higher degree of exploration may result in better AIQ at lower episode lengths, and lower rates of exploration as well as lower learning rates may benefit Q-learning in the long-term.

The final practical goal was to implement a script capable of running the entire suite of analyses on new AIQ test data. Although not optimised, the script fulfils its intended function and can be refined as part of further work in this area.

In conclusion, the thesis achieves its core objectives: it clarifies how various Q-learning configurations perform within the AIQ test using multiple complementary analytical methods and offers a basis for further research into hyperparameter influence on performance.

List of references

- Anaconda Software Distribution. (2020). *Anaconda Documentation*. Anaconda Inc. Retrieved from <https://docs.anaconda.com/>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.
- Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *PeerJ Computer Science*, 7*, e623. <https://doi.org/10.7717/peerj-cs.623>
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems* (3rd ed.). O'Reilly Media, Inc.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hernández-Orallo, J. (2017). *The measure of all minds: Evaluating natural and artificial intelligence*. Cambridge University Press. <https://doi.org/10.1017/9781316594179>
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), 90–95. <https://matplotlib.org>
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., ... Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583–589. <https://doi.org/10.1038/s41586-021-03819-2>
- Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4, 237–285. <https://doi.org/10.1613/jair.301>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). *Jupyter Notebooks – A publishing format for reproducible computational workflows*. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>

- Legg, S., & Hutter, M. (2007a). Universal intelligence: A definition of machine intelligence. *Minds and Machines*, 17(4), 391–444. <https://doi.org/10.1007/s11023-007-9079-x>
- Legg, S., & Hutter, M. (2007b). A collection of definitions of intelligence. *Frontiers in Artificial Intelligence and Applications*, 157, 17.
- Legg, S., & Veness, J. (2011). An empirical metric for intelligence. In J. Schmidhuber, K. R. Thorisson, & M. Looks (Eds.), *Artificial General Intelligence: Proceedings of the 4th International Conference on AGI* (pp. 90–101). Springer. https://doi.org/10.1007/978-3-642-22887-2_10
- Makridakis, S. (2017). The forthcoming artificial intelligence (AI) revolution: Its impact on society and firms. *Futures*, 90, 46–60. <https://doi.org/10.1016/j.futures.2017.03.006>
- Masa, P., & Little Big Company, s.r.o. (2025). *CleverMiner documentation*. <https://www.cleverminer.org/doc/index.html>
- Merchant, A., Batzner, S., Schoenholz, S. S., Aykol, M., Cheon, G., & Cubuk, E. D. (2023). Scaling deep learning for materials discovery. *Nature*, 624(7990), 80–85. <https://doi.org/10.1038/s41586-023-06694-z>
- Mitchell, T. (1997). *Machine Learning*. New York, NY: McGraw-Hill Professional.
- OpenAI. (2023). *GPT-4 technical report* (No. arXiv:2303.08774). arXiv. <https://arxiv.org/abs/2303.08774>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org>
- Python Software Foundation. (2024). *Python language reference* (Version 3.12.7) [Computer software]. <https://www.python.org>
- Raven, J. (2003). Raven progressive matrices. In R. McCallum (Ed.), *Handbook of nonverbal assessment* (pp. 223–237). Springer US. https://doi.org/10.1007/978-1-4615-0153-4_11
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., ... Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144. <https://doi.org/10.1126/science.aar6404>

Solomonoff, R. J. (1964). A formal theory of inductive inference: Part 1 and part 2. *Information and Control*, 7, 1–22, 224–254. [https://doi.org/10.1016/S0019-9958\(64\)90223-2](https://doi.org/10.1016/S0019-9958(64)90223-2)

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. (Original work published 1998). <https://web.archive.org/web/20250221061524/http://incompleteideas.net/book/RLbook2020.pdf>

The pandas development team. (2020). *pandas-dev/pandas: Pandas* [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.3509134>

Vadinský, O., & Zeman, P. (2024). Towards evaluating policy optimisation agents using algorithmic intelligence quotient test. In S. Nowaczyk et al. (Eds.), *Artificial Intelligence. ECAI 2023 International Workshops. ECAI 2023. Communications in Computer and Information Science* (Vol. 1947). Springer. https://doi.org/10.1007/978-3-031-50396-2_25

Watkins, C. J. C. H. (1989). *Learning from delayed rewards* [Doctoral dissertation, University of Cambridge].

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292. <https://doi.org/10.1007/BF00992698>

Witten, I. H., & Frank, E. (2005). *Data mining: Practical machine learning tools and techniques* (2nd ed.). Morgan Kaufmann.

Appendix

The full set of the analyses' scripts, results and a sample of published AIQ test data are available in a GitLab repository:

<https://gitlab.com/Dlom0101/aiq-test-agent-performance-analysis/-/tree/8817d254b338f5961be95894c160ff41a923b9a9/>

The repository contains:

- Script for the analyses performed in the thesis
- Jupyter Notebook equivalent of the script
- EDA result files
- Demonstrational dataset for the script's testing